

SubLogarithmic Linear Time SNARKs from Improved Sumcheck

Sikhar Patranabis¹, Nitin Singh², and Sayani Sinha³

^{1,2}IBM Research India

³IIT Kharagpur, India

Abstract

We present **HybridSpartan** and **HybridPlonk** – the first SNARKs that simultaneously achieve linear-time prover, sublogarithmic proof size, logarithmic verifier, and also feature updatable setups. Our constructions are provably secure in the random oracle model. As a core technical contribution (possibly of independent interest), we reduce the communication complexity of the classical sumcheck protocol for multivariate polynomials from logarithmic to sublogarithmic, while retaining linear prover complexity. For degree d multivariate polynomials in μ variables which can be decomposed into ℓ multilinear polynomials, our protocol achieves $O(\ell + d \log \log n)$ communication and $O(n)$ prover cost for $n = 2^\mu$. Our protocol leverages recently proposed multilinear polynomial commitment schemes (PCS) with linear-time prover and constant proof size.

Multivariate sumcheck is a key ingredient in the design of several prover-efficient SNARKs, such as Spartan (Crypto '20), HyperPlonk (Eurocrypt '23), MicroSpartan (S&P '25), Hyrax (S&P '18), Libra (Crypto '19), Gemini (Eurocrypt '22), Virgo (S&P '20) etc. All of these SNARKs incur $\Omega(\log n)$ proof size, with the smallest concrete proof sizes ranging from 5KB-10KB for circuit sizes in the range of 2^{20} - 2^{30} .

We compile variants of the Spartan and HyperPlonk multilinear PIOPs with our improved sumcheck to realize two new SNARKs, namely **HybridSpartan** and **HybridPlonk**, that support R1CS and Plonkish constraints, respectively. Both of them achieve $O(n)$ prover, $O(\log \log n)$ proof size, and $O(\log n)$ verifier, while avoiding proof recursion and non-black-box use of cryptographic primitives. We implement **HybridSpartan** and **HybridPlonk**, and compare their performances with several state-of-the-art prover-efficient SNARKs. For circuits of size 2^{30} , **HybridSpartan** and **HybridPlonk** achieve proof sizes of 1.8 KB and 2.3 KB, respectively, which are $2.5 - 3.8\times$ smaller than the most compact prover-efficient SNARKs, while achieving comparable prover costs.

Contents

1	Introduction	3
1.1	Our Contributions	4
1.2	Comparison with Related Work	6
2	Technical Overview	8
2.1	Improved Sumcheck Argument with Sublogarithmic Rounds	9
2.2	Compiling to SNARKs	12
3	Preliminaries	12
3.1	Succinct Argument of Knowledge	12
3.2	Polynomial Commitment Scheme	13
3.3	The KZG PCS	14
3.4	Polynomial IOP	15
3.5	Multivariate Sumcheck	16
4	Sumcheck with Sublogarithmic Communication	17
4.1	Using a Univariate PIOP	17
4.2	Checking Consistency of Polynomials	19
4.3	Communication-optimized End-to-End Protocol	20
4.4	Putting Everything Together	24
5	HybridSpartan	25
5.1	Background: MicroSpartan	25
5.2	HybridSpartan via Improved Sumcheck	27
5.3	Concrete Proof Size of HybridSpartan	27
6	HybridPlonk	28
6.1	Background: HyperPlonk	28
6.2	HybridPlonk via Improved Sumcheck	29
6.3	Concrete Proof Size of HybridPlonk	30
7	Implementation and Evaluation	30
8	Extensions and Future Work	33

1 Introduction

A Succinct Non-interactive ARgument of Knowledge (SNARK) allows a prover to convince a verifier about the integrity of a computation such that the size of the proof and the work done by the verifier in checking the proof is much smaller than the size of the computation. Concretely, for an NP relation $\mathcal{R}$, a SNARK proof π corresponding to a statement $\mathfrak{x}$ convinces a verifier (non-interactively) about the existence of witness $\mathfrak{w}$ such that $\mathcal{R}(\mathfrak{x}, \mathfrak{w}) = 1$. Succinctness requires that the size of π and the verification complexity are both $O_\lambda(\text{polylog}(|\mathfrak{x}| + |C|))$, where $|\mathfrak{x}|$ is the size of the statement $\mathfrak{x}$, $|C|$ is the number of gates in the circuit C that verifies the relation $\mathcal{R}(\mathfrak{x}, \mathfrak{w}) = 1$, and λ is the security parameter¹. While initially introduced as theoretical concepts in the seminal contributions of Micali [Mic94] and Kilian [Kil92], SNARKs have since been refined in a long line of works [Gro10, Lip12, BCCT12, BCI⁺13, GGPR13, PHGR13, BCG⁺13, Lip13, BCTV14] towards practical adoption. More recently, SNARKs have seen real-world deployment in several niche applications such as decentralized payment systems like Zcash [BCG⁺14] and Monero [NMT], decentralized file storage systems like Filecoin [Lab17], blockchain scaling solutions such as ZkRollups [rol21], using existing credentials for authentication on blockchain [BCJ⁺24], and ensuring privacy and integrity of machine learning applications [LVA⁺25, LKKO20, ZFZS20, SDP22].

Models for SNARKs. Efficient SNARKs typically require a Structured Reference String (SRS) to be generated at setup. Setup for SNARKs in the SRS model can be broadly classified into three categories: (i) *transparent* (setup only uses public randomness and verifier randomness consists of only *public coins*), (ii) *trusted* (require a randomized, circuit-dependent preprocessing phase with secret random coins), and (iii) *universal and updatable* (a *one-time* setup can be used to prove statements about any computation; moreover, there is a mechanism to update by contributing to the randomness of the SRS, and the SRS is trusted as long as at least one of the updates is honest). While the first practical SNARKs were in the trusted SRS model [PHGR13, GGPR13], several recent (pairing-based) SNARKs are in the updatable SRS model [MBKM19, GWC19, CHM⁺20, Set20, BCHO22, CBBZ23, XZZ⁺19, ZSCZ25, GPS25], which is also the model that we use in this paper.

SNARKs from PIOP and PCS. A common design methodology underlying several modern SNARKs [WTs⁺18, GWC19, XZZ⁺19, CHM⁺20, Set20, SL20, BCHO22, XZS22, CBBZ23, GLS⁺23] is to compile an information-theoretically secure Polynomial Interactive Oracle Proof (PIOP) using a cryptographic tool called a Polynomial Commitment Scheme (PCS). In a PIOP, the prover provides the verifier with oracle access to a set of polynomials, and the verifier queries these polynomial oracles at random challenge points to obtain evaluations, eventually outputting accept or reject. Compiling a PIOP using a PCS yields a public-coin succinct argument, where the polynomial oracles are realized via *commitments* to polynomials that a verifier can query for evaluations, together with proofs that the evaluations are indeed consistent with the commitments. The interactive argument is further compiled using Fiat-Shamir [FS87] to obtain a SNARK in the Random Oracle Model (ROM).

Univariate and Multilinear PCS. SNARKs based on the above approach can be broadly classified into two categories: those realized by compiling univariate PIOPs using univariate PCS (such

¹Throughout the paper, all of our asymptotics are stated for growing instance size, and have implicit dependence on a fixed security parameter λ . Hence, we simply write $O(\cdot)$ instead of $O_\lambda(\cdot)$ for simplicity of exposition.

as Sonic [MBKM19], Marlin [CHM⁺20], Plonk [GWC19], and Pari [DMS24]), and those realized by compiling multilinear PIOPs using multilinear PCS (such as Hyrax [WTs⁺18], Libra [XZZ⁺19], Spartan [Set20], Kopis [SL20], Xiphos [SL20], Gemini [BCHO22], Orion [XZS22], Brakedown [GLS⁺23] and HyperPlonk [CBBZ23]). Univariate PCS-based SNARKs achieve constant proof size and constant verification complexity. However, their prover time is $O(n \log n)$, where n is the size of the computation (often represented by the size of the circuit or the number of R1CS constraints). This becomes a bottleneck for applications requiring large prover computations [rol21, Lab17], and has sparked increased interest in designing *prover-efficient* SNARKs.

Prover-Efficient SNARKs. SNARKs based on multilinear PIOPs and multilinear PCS [WTs⁺18, XZZ⁺19, Set20, SL20, CBBZ23, BCHO22, XZS22, GLS⁺23] are prover-efficient in the sense that they achieve $O(n)$ prover. However, they incur $O(\log n)$ proof size. With growing computation sizes reaching $\approx 2^{30}$, even the best known constructions incur concrete proof sizes ≈ 8 -10 KB ². In comparison, Plonk [GWC19] achieves constant proof size of 0.61 KB. Testudo [CGG⁺23] achieves constant proof size by recursively composing sumcheck verification and inner pairing-product verification with a Groth16 [Gro16] SNARK. However, such a recursive composition requires non-native operations (such as target group scalar multiplications) to be encoded inside arithmetic circuits which makes it practically challenging, limits the choice of elliptic curves to instantiate the scheme, and prevents the possibility of proving security in the ROM. In this paper, we ask the following question:

Can we design a SNARK in the ROM with linear-time prover and sublogarithmic proof size?

Multivariate Sumcheck. Several prover-efficient SNARKs [WTs⁺18, XZZ⁺19, Set20, ZXZS20, BCHO22, CBBZ23] are designed using PIOPs for *multivariate sumcheck*. These schemes inherit their logarithmic proof size from the logarithmic communication complexity of the classical sumcheck argument for multivariate polynomials [LFKN90]. Consequently, though recent works [GPS25, EG25] have proposed multilinear PCS with linear-time prover and constant proof size ³, they cannot be used to compile multivariate sumcheck PIOPs to yield prover-efficient SNARKs with sublogarithmic proof size. The communication complexity of multivariate sumcheck is thus a major barrier towards achieving prover-efficient SNARKs with sublogarithmic proof size. This leads to the following question:

Can we design a multivariate sumcheck argument with linear-time prover and sublogarithmic communication?

1.1 Our Contributions

We answer both of the above questions in the affirmative. We exhibit a new multivariate sumcheck argument with linear-time prover and sublogarithmic communication. We then leverage this protocol to design two SNARKs – **HybridSpartan** (for R1CS constraints) and **HybridPlonk** (for Plonkish constraints) – the first SNARKs in the updatable SRS setting that simultaneously achieves $O(n)$ prover time, sublogarithmic proof size of $O(\log \log n)$, and $O(\log n)$ verifier, while also being provably secure in the ROM. We expand on our contributions below.

²Throughout this paper, we report proof sizes assuming a BLS12-381 curve-based implementation, unless specified otherwise.

³We consider multiexponentiation of size n to be $O(n)$ work, with implicit dependence on security parameter λ .

Table 1: Comparison of pairing-based SNARKs with linear-time prover. Here, n denotes the number of R1CS constraints (or the number of gates in a circuit), and $(\mathbb{F}, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T)$ denote the field and groups underlying a bilinear pairing. $\mathbb{P}$ denotes a pairing evaluation. All concrete proof sizes are reported with respect to the BLS12-381 curve, except MicroSpartan [ZSCZ25] and HybridSpartan, which are reported for the BN254 curve. The concrete proof size for Libra depends on the circuit depth d .

SNARK	Prover Time	Verifier Time	Proof Size	$n = 2^{20}$	Setup
Dory [Lee21]	$O(n) \mathbb{G}_1$	$O(\log n) \mathbb{G}_T$	$O(\log n) \mathbb{G}_T$	24KB	Transparent
Kopis [SL20]		$O(\sqrt{n}) \mathbb{G}_T$	$O(\log n) \mathbb{G}_T$	39KB	
Xiphos [SL20]		$O(\log n) \mathbb{G}_T$	$O(\log n) \mathbb{G}_T$	61KB	
Garuda [DMS24]		$O(\log n) \mathbb{P}$	$O(\log n) \mathbb{G}_1$	$\approx 10\text{KB}$	Trusted
Spartan [Set20] + KZG [KZG10]		$O(\log^2 n) \mathbb{F}, O(\log n) \mathbb{G}_1$	$O(\log^2 n) \mathbb{F}$	234KB	
Gemini [BCHO22]		$O(\log n) \mathbb{G}_1$	$O(\log n) \mathbb{G}_1$	18KB	Updatable
HyperPlonk [CBBZ23] + PST [PST13]		$O(\log n \log(\log n)) \mathbb{F}, O(\log n) \mathbb{P}$	$O(\log n) \mathbb{G}_1$	5.5KB	
HyperPlonk + HyperKZG [BCHO22]		$O(\log n \log(\log n)) \mathbb{F}, O(\log n) \mathbb{G}_1$	$O(\log n) \mathbb{G}_1$	6.1KB	
Libra [XZZ ⁺ 19]		$O(d \log n) \mathbb{F}, O(\log n) \mathbb{G}_1$	$O(d \log n) \mathbb{G}_1$	–	
MicroSpartan [ZSCZ25] (BN254)		$O(\log n) \mathbb{G}_1$	$O(\log n) \mathbb{G}_1$	5KB	
Samaritan [GPS25]		$O(\log n) \mathbb{F}, O(1) \mathbb{G}_1$	$O(\log n) \mathbb{F}, O(1) \mathbb{G}_1$	6.2KB	
HyperPlonk + Samaritan PCS [GPS25]		$O(\log n \log(\log n)) \mathbb{F}, O(\log n) \mathbb{G}_1$	$O(\log n) \mathbb{F}, O(1) \mathbb{G}_1$	4.2KB	
HybridSpartan (this work, BN254)		$O(\log n) \mathbb{F}, O(\log n) \mathbb{G}_1$	$O(\log \log n) \mathbb{F}, O(1) \mathbb{G}_1$	1.8KB	
HybridPlonk (this work)		$O(\log n) \mathbb{F}, O(\log n) \mathbb{G}_1$	$O(\log \log n) \mathbb{F}, O(1) \mathbb{G}_1$	2.3KB	

Improved Multivariate Sumcheck. We leverage two recent works [GPS25, EG25] that have proposed multilinear polynomial commitment schemes to reduce the communication complexity of the classical sumcheck argument for multivariate polynomials [LFKN90]. For degree d multivariate polynomials in μ variables which can be decomposed into ℓ multilinear polynomials, we exhibit a new multivariate sumcheck argument with $O(\ell + d \log \log n)$ communication for $n = 2^\mu$. As described in Section 4, the proposed argument retains the $O(n)$ prover cost (where the precise constant depends on ℓ , d and the multivariate form). Thus we improve on the $O(\log n)$ communication inherent in all existing applications of the multivariate sumcheck argument.

Sublogarithmic Linear Time SNARKs. We apply our improved sumcheck argument to obtain the *first* linear-time prover SNARKs with sublogarithmic proof sizes supporting either R1CS or Plonkish constraints:

- In Section 5, we compile recent improvements to the Spartan multilinear PIOP [Set20, ZSCZ25] with our improved sumcheck argument to realize **HybridSpartan**, that supports R1CS constraints.
- In Section 6, we compile an optimized variant of the HyperPlonk [CBBZ23] multilinear PIOP with our improved sumcheck argument to realize **HybridPlonk**, that supports Plonkish constraints.

Both SNARKs feature *updatable setup* and *sublogarithmic proof size* of $O(\log \log n)$, while preserving $O(n)$ prover time and $O(\log n)$ verifier. They also avoid proof recursion and non-black-box use of cryptographic primitives, and can be proven secure in the ROM.

We note that the asymptotic improvements to the proof sizes of **HybridSpartan** and **HybridPlonk** are significant at instance sizes of practical interest, without sacrificing prover and verifier efficiency substantially. Below, we compare the proof sizes of **HybridSpartan** and **HybridPlonk** with those for MicroSpartan [ZSCZ25] and HyperPlonk compiled with Samaritan PCS [GPS25], which are the state of the art prover-efficient SNARKs with the smallest proof sizes:

- For **HybridSpartan**, the proof size over the BN254 curve is 1.81KB at circuit sizes of 2^{30} . This is $\approx 3.8\times$ smaller than the proof size of Microspartan [ZSCZ25] over BN254 for the same circuit size ⁴.
- For **HybridPlonk**, the proof size over the BLS12-381 curve is 2.34KB at circuit sizes of 2^{30} . This is $\approx 2.5\times$ smaller than the proof size of HyperPlonk compiled with Samaritan PCS [GPS25] over BLS12-381 for the same circuit size.

Note that SNARKs with smaller proof sizes than **HybridSpartan** and **HybridPlonk** are based on univariate PCS, and are not prover-efficient as they inherently incur $O(n \log n)$ prover cost due to polynomial multiplications. We present a detailed comparison of **HybridSpartan** and **HybridPlonk** with existing SNARKs in Section 1.2 and Table 1.

Evaluation. We implement **HybridSpartan** and **HybridPlonk** (as well as Samaritan PCS [GPS25]), on top of the popular Arkworks [ac22] ecosystem. Our implementation is available at: <https://github.com/SayaniSinha97/HybridSNARK>. We provide a thorough empirical evaluation of both **HybridSpartan** and **HybridPlonk**, and compare them with relevant recent works in Section 7.

Extension to zkSNARK. Zero-knowledge, while a desirable guarantee, is not the primary focus of this work. While we focus on succinctness in this paper, we believe it should be possible to achieve zero-knowledge variants of **HybridSpartan** and **HybridPlonk** following standard techniques in the literature (such as [CHM⁺20, CBBZ23]), albeit with some additional prover cost.

1.2 Comparison with Related Work

Pairing-based Prover-Efficient SNARKs. Table 1 presents a detailed comparison of both **HybridSpartan** and **HybridPlonk** against known pairing-based SNARKs with linear prover time. **HybridSpartan** and **HybridPlonk** are the only ones with sublogarithmic proof sizes, and also achieve the smallest concrete proof sizes. Dory [Lee21], Kopis [SL20], and Xiphos [SL20] support transparent setup (but incur significantly larger proof size and verifier overheads compared to **HybridPlonk**), while Garuda [DMS24] requires a trusted, circuit-dependent setup (unlike **HybridPlonk**, where setup is circuit-independent). All of the other SNARKs in Table 1 are in the updatable setup setting (same as **HybridSpartan** and **HybridPlonk**).

Non-Pairing-based Prover-Efficient SNARKs. Several works have proposed (plausibly post-quantum secure) SNARKs with transparent setup and linear prover from hash-based assumptions (e.g., FRI [BBHR18], Orion [XZS22], Brakedown [GLS⁺23], BaseFold [ZCF24], STIR [ACFY24], WHIR [ACFY25], and Blaze [BCF⁺25b]) and lattice-based assumptions (e.g., LaBRADOR [BS23], Greyhound [NS24] and [CMNW24]). Asymptotically, all of these constructions incur superlogarithmic proof sizes and verification overheads. In comparison, **HybridPlonk** incurs sublogarithmic proof size and logarithmic verification overhead (but is not post-quantum due to its reliance on pairings). The concrete proof sizes of known hash-based SNARKs [BBHR18, XZS22, GLS⁺23, ZCF24, ACFY24, ACFY25, BCF⁺25b] are greater than 200KB for $n = 2^{20}$ (two orders of magnitude larger than the proof sizes of **HybridSpartan** and **HybridPlonk**). The concrete proof sizes of state-of-the-art lattice-based SNARKs [BS23, NS24, CMNW24] are around 50KB for $n = 2^{20}$ (nearly

⁴The authors of [ZSCZ25] reported proof sizes for Microspartan over the BN254 curve. Hence, we also report proof sizes for **HybridSpartan** over BN254 for a fair comparison.

20 $\times$ larger than HybridSpartan and HybridPlonk). More crucially, their verifier overhead is $O(\sqrt{n})$, which is substantially larger (both asymptotically and concretely) than that of HybridSpartan and HybridPlonk.

SNARKs with SubLogarithmic Proof Size. Several early SNARK constructions [Gro10, Lip12, BCCT12, BCI⁺13, GGPR13, PHGR13, BCG⁺13, Lip13, BCTV14] have constant proof size and verification complexity. These are realized via linear PCPs compiled using cryptographic tools such as linear-only encodings (e.g., exponentiation in a bilinear group). All of these SNARKs incur at least $O(n \log n)$ prover time. More recently, SNARKs with constant proof size and verification complexity have been achieved by compiling univariate PIOPs using univariate PCS. Some prominent examples are Sonic [MBKM19], Marlin [CHM⁺20], Plonk [GWC19], and Pari [DMS24]. These incur $O(n \log n)$ prover time. HybridSpartan and HybridPlonk belong to an alternative family of SNARKs that trade off proof size (and verification time) for more efficient $O(n)$ proof generation. In particular, while the aforementioned SNARKs have concretely smaller proof sizes and verification time than HybridSpartan and HybridPlonk, they incur significantly larger prover overheads.

Recent Improvements to Sumcheck. A recent work [LMN25] proposed a multivariate sumcheck PIOP with $O(\log \log n)$ rounds and $O(n)$ prover work. However, they use Zeromorph [KT24] PCS to compile their PIOP into an argument, which results in a logarithmic sized argument. They do not report compilation to a generic SNARK. Since, the PIOP in [LMN25] involves multivariate polynomials of degree > 1 , it is not immediately clear if the recent multilinear PCS such as Samaritan [GPS25] or Mercury [EG25] can be used to compile their PIOP to an argument.

Concurrent to our work, two recent works [ARZR25, BCF⁺25a] propose $O(\log^* n)$ -round ⁵ multilinear sumcheck PIOPs with $O(n)$ prover work. While [ARZR25] works specifically over binary fields, [BCF⁺25a] generalizes to fields of large prime order. However, these works do not describe full-fledged SNARK constructions based on the corresponding sumcheck PIOPs. In particular, we note that the PIOP in [ARZR25] does not extend to products of polynomials (which is necessary for a full-fledged SNARK). Further, the PIOP in [ARZR25] needs to be compiled using a *multivariate* PCS. This precludes the use of recent multilinear PCS such as Samaritan [GPS25] or Mercury [EG25] to achieve a combination of sublogarithmic proof sizes and linear prover (which is what we achieve in HybridSpartan and HybridPlonk).

The authors of [BCF⁺25a] only briefly sketch how one could compile their sumcheck protocol for products of multilinear polynomials using PST [PST13]-like commitments to achieve an argument system with $O(\log^* n)$ proof size and $O(n)$ prover. However, the $O(n)$ prover is only presented for sumcheck instances over products of two multilinear polynomials. It is not clear if their techniques for obtaining a linear-time prover extend to more general sumcheck instances with multiplicative depth greater than 2 (which is necessary for full-fledged SNARKs).

While it is possible that the approach of [BCF⁺25a] could yield a practical SNARK (perhaps with a slightly superlinear prover), it would have the following limitations: (i) the use of PST [PST13]-like commitments would necessitate a *trusted setup* (and, hence, would not support *updatable setup*), (ii) the setup size and time required for their approach would be slightly superlinear, i.e., $O(n \log^* n)$, which concretely leads to 4-5 $\times$ larger setup, and (iii) verification would require $O(\log^* n)$ pairing checks instead of $O(1)$. On the other hand, our approach is compatible with state-of-the-art multilinear PCS with updatable setup, requires 4-5 $\times$ smaller setup param-

⁵The concrete value of both $\log^* n$ and $\log(\log n)$ (rounded to the nearest integer) is 5 for $n = 2^{32}$.

ters, and features an efficient verifier with $O(1)$ pairings. We leave a detailed construction of a practical SNARK using the techniques of [BCF⁺25a] as interesting future work.

2 Technical Overview

In this section, we present some notational background and a high-level overview of our techniques. Detailed preliminaries on SNARKs are discussed in Section 3.

Notation. We denote the set of integers $\{1, \dots, n\}$ by $[n]$ for $n \in \mathbb{N}$, and $\mathbb{F}$ to denote a prime field of order p . We denote by λ a security parameter. We use negl to denote a negligible function: for any integer $c > 0$, there exists $n \in \mathbb{N}$, such that $\forall x > n$, $\text{negl}(x) \leq 1/x^c$. We assume a bilinear group generator BG which on input λ outputs parameters for the protocols. Specifically $\text{BG}(1^\lambda)$ outputs $(\mathbb{F}, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, g_t)$ where: $\mathbb{F} = \mathbb{F}_p$ is a prime field of super-polynomial size in λ , with $p = \lambda^{\omega(1)}$; $\mathbb{G}_1, \mathbb{G}_2$ and $\mathbb{G}_T$ are groups of order p , and e is an efficiently computable non-degenerate bilinear pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$; Generators g_1, g_2 are uniformly chosen from $\mathbb{G}_1$ and $\mathbb{G}_2$ respectively and $g_t = e(g_1, g_2)$. We write groups $\mathbb{G}_1$ and $\mathbb{G}_2$ additively, and use the shorthand notation $[x]_1$ and $[x]_2$ to denote group elements $x \cdot g_1$ and $x \cdot g_2$ respectively for $x \in \mathbb{F}$. We implicitly assume that all the setup algorithms for the protocols invoke BG to generate descriptions of groups and fields over which the protocol is instantiated. We will use sets $\mathbb{F}, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ to specify the type of operations, where additionally, we have $\mathbb{P}$ to denote pairings and $\mathbb{M}$ to denote multiexponentiation.

Sets. For $\mu \in \mathbb{N}$, we use B_μ to denote the set $\{0, 1\}^\mu$. For $\mathbf{y} = (y_1, \dots, y_\mu) \in B_\mu$, we use the notation $\text{id}_\mu(\mathbf{y})$ to denote the integer $1 + \sum_{i=1}^\mu y_i 2^{i-1}$, and similarly for an integer i , we use $\langle i \rangle_\mu$ to denote the μ -bit binary decomposition of $i - 1$. We note that the surjective maps $\text{id}_\mu : B_\mu \rightarrow [2^\mu]$ and $\langle \cdot \rangle_\mu : [2^\mu] \rightarrow B_\mu$ are inverses of each other. We will drop the subscript μ when it is clear from the context.

Polynomials and Multilinear Extensions. We use $\mathbb{F}^{\leq 1}[X_1, \dots, X_\mu]$ to denote the set of μ -variate multilinear polynomials over the field $\mathbb{F}$. We define 2μ -variate polynomial

$$\tilde{e}q_\mu(\mathbf{x}, \mathbf{y}) = \prod_{i=1}^\mu (x_i y_i + (1 - x_i)(1 - y_i))$$

for $\mathbf{x} \in \mathbb{F}^\mu$ and $\mathbf{y} \in \mathbb{F}^\mu$. The polynomials $\{\tilde{e}q_\mu(x, \langle i \rangle) : i \in [2^\mu]\}$ are linearly independent over $\mathbb{F}$ and form the Lagrange basis polynomials for the set B_μ . For $\mathbf{x}, \mathbf{y} \in B_\mu$, $\tilde{e}q_\mu(\mathbf{x}, \mathbf{y}) = 1$ if $\mathbf{x} = \mathbf{y}$, and is 0 otherwise. For a function $f : B_\mu \rightarrow \mathbb{F}$, the (unique) polynomial $\tilde{f}(\mathbf{x}) = \sum_{i=1}^{2^\mu} f(\langle i \rangle) \tilde{e}q_\mu(\mathbf{x}, \langle i \rangle)$ is called the *multilinear extension* (MLE) of the function f . We also naturally view vectors $\mathbf{f} \in \mathbb{F}^{2^\mu}$ as functions $f : B_\mu \rightarrow \mathbb{F}$, and define MLE of the vector as that of the implied function. For clarity of notation, we will denote multilinear polynomials as $\tilde{f}$ (with a tilde), its associated coefficient vector of evaluations at B_μ as $\mathbf{f}$, and the univariate polynomial with $\mathbf{f}$ as the coefficient vector (in power basis $1, X, \dots, X^{n-1}$ for $n = 2^\mu$) as $\hat{f}(X)$. Thus, the univariate and multilinear polynomials sharing the coefficient vector $\mathbf{f}$ in the respective bases are denoted as $\hat{f}$ and $\tilde{f}$ respectively.

2.1 Improved Sumcheck Argument with Sublogarithmic Rounds

In this subsection, we present an overview of our main technical contribution, namely, an improved sumcheck argument with sublogarithmic communication and linear prover. For the sake of illustration, consider the claim

$$\sum_{\mathbf{x} \in B_\mu} \tilde{a}(\mathbf{x}) \tilde{b}(\mathbf{x}) = v$$

for μ -variate multilinear polynomials $\tilde{a}$ and $\tilde{b}$. Let $n = 2^\mu$. Using the sumcheck argument described in Section 3.5, the prover can prove the above claim in $O(\log n)$ rounds of communication while doing $O(n)$ work. We ask:

Can we obtain a sumcheck argument with fewer communication rounds?

Our Approach. A possible approach is to use the *univariate sumcheck* protocol introduced in [BCR⁺19], where instead of considering multilinear polynomial evaluations over the hypercube, we consider univariate polynomial evaluations over a multiplicative subgroup of $\mathbb{F}$ of size n . Specifically, letting $\mathbb{H} = \{1, \omega, \dots, \omega^{n-1}\}$ be the multiplicative subgroup of $\mathbb{F}$ of size n with $\omega \in \mathbb{F}$ as a generator, we define univariate polynomials $\hat{a}, \hat{b} \in \mathbb{F}^{<n}[X]$ such that $\hat{a}(\omega^{i-1}) = \tilde{a}(\langle i \rangle)$ and $\hat{b}(\omega^{i-1}) = \tilde{b}(\langle i \rangle)$. Then, the equivalent claim to prove is

$$\sum_{h \in \mathbb{H}} \hat{a}(h) \hat{b}(h) = v$$

The univariate sumcheck protocol [BCR⁺19], when used with the KZG PCS allows the identity to be checked using $O(1)$ communication. However, the prover work is $O(n \log n)$ due to the need to compute polynomial product $\hat{a}(X) \cdot \hat{b}(X)$. The next question we ask is:

Can we reduce communication while retaining $O(n)$ prover work?

We give an affirmative answer to this question, achieving communication of $O(\log \log n)$ with linear prover effort. We achieve this by using a hybrid of the two sumcheck techniques. At a high level, our approach is as follows:

- We initially proceed with the multilinear sumcheck protocol for $O(\log \log n)$ rounds, at the end of which we are left with a claim over hypercube of size $m = n / \log n$, involving multilinear polynomials of size m .
- By identifying the multilinear polynomials of size m with univariate polynomials of degree $< m$, we derive an equivalent univariate sumcheck claim.
- Using the univariate sumcheck protocol, and the KZG PCS, we can then prove the claim with $O(1)$ communication and $O(m \log m)$ prover work, which is $O(n)$ since $m = n / \log n$.

The above approach is detailed in Section 4.1.

Challenges. While the above approach gives us a template for a sumcheck argument in $O(\log \log n)$ rounds, we run into two main challenges:

- We need to prove that the multilinear polynomials of size $m = n/\log n$ are suitable restrictions of the original multilinear polynomials of size n .
- We also need to prove that the polynomials involved in the univariate sumcheck are consistent with the multilinear polynomials in the reduced sumcheck instance. This, in turn, requires proving that a pair of polynomials share the same coefficients in the power and Lagrange bases.

We address both challenges by crucially using techniques from Samaritan PCS [GPS25] and Mercury [EG25]. Our approach is described in Sections 4.2 and 4.3. We present a more expanded illustration of our techniques below.

Illustration of Our Approach. We illustrate the approach outlined above for the relation

$$\sum_{\mathbf{x} \in B_\mu} \tilde{a}(\mathbf{x}) \tilde{b}(\mathbf{x}) = v \quad (1)$$

over μ -variate multilinear polynomials $\tilde{a}$ and $\tilde{b}$ over a field $\mathbb{F}$. We choose this relation since it allows for a simpler exposition of our techniques, but, at the same time, involves a polynomial multiplication operation. See Section 4 for a treatment of a general version of the sumcheck relation.

Let $n = 2^\mu$ denote the size of the hypercube. Choose $\rho = \log \log n$, and set $\kappa = \mu - \rho$. Let $m = 2^\kappa$. We observe that $m = n/\log n$. To prove the relation in Equation (1), the prover and verifier initially proceed with the first ρ rounds of the classical multivariate sumcheck protocol described in Section 3.5. Assuming $\mathbf{r} = (r_1, \dots, r_\rho)$ is the set of challenges sent by the verifier so far, the reduced relation is of the form

$$\sum_{\mathbf{x} \in B_\kappa} \tilde{a}(\mathbf{r}, \mathbf{x}) \tilde{b}(\mathbf{r}, \mathbf{x}) = \bar{v} \quad (2)$$

for some $\bar{v} \in \mathbb{F}$. At this stage, the prover commits to κ -variate multilinear polynomials $\tilde{p}_1$ and $\tilde{p}_2$ defined by $\tilde{p}_1(\mathbf{x}) = \tilde{a}(\mathbf{r}, \mathbf{x})$ and $\tilde{p}_2(\mathbf{x}) = \tilde{b}(\mathbf{r}, \mathbf{x})$, for $\mathbf{x} \in B_\kappa$. Let $\hat{p}_1(X)$ and $\hat{p}_2(X)$ denote the corresponding univariate polynomials of degree less than m , such that j^{th} coefficient of $\hat{p}_i$ is $\tilde{p}_i(\langle j \rangle)$, $\forall i \in \{1, 2\}, j \in [m]$.

Using a Univariate PIOP. We verify the reduced relation (Equation 2) using a univariate PIOP over a much smaller, m -sized domain $\mathbb{H}$ consisting of the m roots of unity $\{1, \omega, \dots, \omega^{m-1}\}$. Concretely, the prover computes polynomials $\hat{q}_1(X)$ and $\hat{q}_2(X)$, such that evaluations of $\hat{q}_1$ (resp., $\hat{q}_2$) over the set $\mathbb{H}$ match the monomial coefficients of polynomial $\hat{p}_1$ (resp., $\hat{p}_2$). Let $\{\mu_i^\mathbb{H}(X)\}_{i \in [m]}$ denote the Lagrange basis polynomials for the set $\mathbb{H}$, and let ifft be a linear operator on the set $\mathbb{F}^{<m}[X]$ defined as

$$\text{ifft} \left(\sum_{i=1}^m a_i X^{i-1} \right) = \sum_{i=1}^m a_i \mu_i^\mathbb{H}(X).$$

In terms of the above operator, we have $\hat{q}_i = \text{ifft}(\hat{p}_i)$ for $i \in \{1, 2\}$. Assuming that the polynomials $\hat{p}_i$ and $\hat{q}_i$ are honest, we can reduce the sumcheck relation in Equation (2) to a univariate sumcheck relation as below:

$$\sum_{\mathbf{x} \in B_\kappa} \tilde{p}_1(\mathbf{x}) \tilde{p}_2(\mathbf{x}) = \bar{v} \Leftrightarrow \sum_{i \in [m]} \tilde{p}_1(\langle i \rangle) \tilde{p}_2(\langle i \rangle) = \bar{v} \Leftrightarrow \sum_{i \in [m]} \hat{q}_1(\omega^{i-1}) \hat{q}_2(\omega^{i-1}) = \bar{v}, \quad (3)$$

where the second equivalence uses $\hat{q}_i(\omega^{j-1}) = \tilde{p}_i(\langle j \rangle)$ for all $i \in \{1, 2\}$ and $j \in [m]$. In Section 4.1, we show that, using a KZG-based *univariate sumcheck* argument, proving the above relation in Equation 3 incurs $O(1)$ communication and $O(m \log m) = O(n)$ prover work (since $m = n/\log n$).

Checking Consistency of Polynomials. Next, we need to prove that $(\hat{p}_1, \hat{p}_2)$ and $(\hat{q}_1, \hat{q}_2)$ are correctly computed from the original polynomials $\tilde{a}$ and $\tilde{b}$, and the sumcheck challenges $\mathbf{r} = (r_1, \dots, r_\rho)$. We do this in two steps. First, we prove that $\tilde{p}_1(\cdot) = \tilde{a}(\mathbf{r}, \cdot)$ and $\tilde{p}_2(\cdot) = \tilde{b}(\mathbf{r}, \cdot)$. In Section 4.2, we show how to do this with $O(1)$ communication and $O(m+n)$ prover cost, using the recent multilinear PCS with $O(1)$ opening proof sizes such as Samaritan [GPS25] and Mercury [EG25] (discussed in Section 3.5).

Next, we need to prove that $\hat{q}_i = \text{ifft}(\hat{p}_i)$ for each $i \in \{1, 2\}$. In Section 4.2, we show that, using the linearity of ifft operation, we can batch the above checks into a single check of the form $\hat{q} = \text{ifft}(\hat{p})$ where $\hat{q}$ (resp., $\hat{p}$) is a linear combination of $\hat{q}_1$ and $\hat{q}_2$ (resp., $\hat{p}_1$ and $\hat{p}_2$). So, this effectively reduces to proving that $\hat{p}$ and $\hat{q}$ share the same coefficients in the power basis and the Lagrange basis, respectively. At this point, we make the following key observation: suppose that $\hat{q} = \text{ifft}(\hat{p})$. Then, we must have

$$\hat{q}(z) = \text{ifft}(\hat{p}(z)) = \sum_{j=1}^m p_j \mu_j^{\mathbb{H}}(z),$$

where p_j is the j^{th} coefficient of $\hat{p}$, and

$$\mu_j^{\mathbb{H}}(z) = \frac{\omega^{j-1}(z^m - 1)}{m(z - \omega^{j-1})}.$$

This allows us to reduce the claim $\hat{q}(X) = \text{ifft}(\hat{p}(X))$ to the following inner product argument for a random challenge z sent across by the verifier:

$$\begin{aligned} \sum_{j=1}^m p_j \cdot \frac{\omega^{j-1}(z^m - 1)}{m(z - \omega^{j-1})} &= \hat{q}(z) = \sum_{i=1}^{\ell} \gamma^{i-1} \hat{q}_i(z) = v, \\ \implies \sum_{j=1}^m p_j \cdot \frac{\omega^{j-1}}{z - \omega^{j-1}} &= \frac{mv}{z^m - 1} \\ \implies \langle \hat{p}, \hat{A}_z \rangle &= \frac{mv}{z^m - 1}. \end{aligned}$$

where

$$\hat{A}_z(X) = \sum_{j=1}^m \frac{\omega^{j-1}}{z - \omega^{j-1}} X^{j-1}$$

and where, for any $a(X) = \sum_{j=1}^m a_j X^{j-1}$, $b(X) = \sum_{j=1}^m b_j X^{j-1}$, we use $\langle a, b \rangle$ to denote the value $\sum_{i=1}^m a_i b_i$. Here, γ is a random batching challenge sent by the verifier. We now need to prove the following:

- The inner product relation $\langle \hat{p}, \hat{A}_z \rangle = \frac{mv}{z^m - 1}$.
- $\hat{A}_z$ is correctly constructed.

In Section 4.3, we present arguments for both of the above relations that incur $O(1)$ communication and $O(m \log m) = O(n)$ prover work (since $m = n / \log n$).

Putting Everything Together. We now put together the above techniques to state the following (informal) theorem for our improved sumcheck argument.

Theorem 2.1 (Restated formally in Theorem 4.1). *Assuming the security of the KZG univariate PCS [KZG10], there exists an argument of knowledge for the relation in Equation 1 where: (i) the proof size is $O(\log \log n)$, (ii) the prover incurs $O(n)$ work, and (iii) the verifier incurs $O(\log n)$ work.*

2.2 Compiling to SNARKs

HybridSpartan. In Section 5, we show how to compile the MicroSpartan multilinear PIOP [ZSCZ25] with our improved sumcheck argument and Samaritan PCS [GPS25]. The resulting SNARK, which we call HybridSpartan, supports R1CS constraints, and achieves $O(n)$ prover, $O(\log \log n)$ proof size, and $O(\log n)$ verifier, while avoiding proof recursion and non-black-box use of cryptographic primitives (see Section 5.2 for a detailed analysis). Our result is captured by the following theorem.

Theorem 2.2 (HybridSpartan). *Assuming the security of the KZG univariate PCS [KZG10], there is a SNARK that satisfies completeness and knowledge-soundness in the random oracle model, and allows proving the satisfiability of $O(n)$ R1CS constraints such that: (i) the proof size is $O(\log \log n)$, (ii) the prover effort is $O(n)$, and (iii) the verifier effort is $O(\log n)$.*

HybridPlonk. In Section 6, we show how to compile the HyperPlonk multilinear PIOP [CBBZ23] with our improved sumcheck argument and Samaritan PCS. The resulting SNARK, which we call HybridPlonk, supports Plonkish constraints, and achieves the same asymptotic efficiency and cryptographic properties as HybridSpartan (see Section 6.2 for a detailed analysis). Our result is captured by the following theorem.

Theorem 2.3 (HybridPlonk). *Assuming the security of the KZG univariate PCS [KZG10], there is a SNARK that satisfies completeness and knowledge-soundness in the random oracle model, and allows proving the satisfiability of $O(n)$ Plonkish constraints such that: (i) the proof size is $O(\log \log n)$, (ii) the prover effort is $O(n)$, and (iii) the verifier effort is $O(\log n)$.*

3 Preliminaries

3.1 Succinct Argument of Knowledge

Let $\mathcal{R}$ be a NP-relation and $\mathcal{L}$ be the corresponding NP-language, where $\mathcal{L} = \{x : \exists w \text{ such that } (x, w) \in \mathcal{R}\}$. Here, a prover $\mathcal{P}$ aims to convince a verifier $\mathcal{V}$ that $x \in \mathcal{L}$ by proving that it knows a

witness w for a public statement x such that $(x, w) \in \mathcal{R}$. An interactive argument of knowledge for a relation $\mathcal{R}$ consists of a PPT algorithm **Setup** that takes as input the security parameter λ , and outputs the public parameters $\mathbf{pp}$, and a pair of interactive PPT algorithms $\langle \mathcal{P}, \mathcal{V} \rangle$, where $\mathcal{P}$ takes as input $(\mathbf{pp}, x, w)$ and $\mathcal{V}$ takes as input $(\mathbf{pp}, x)$. An interactive argument of knowledge $\langle \mathcal{P}, \mathcal{V} \rangle$ must satisfy completeness and knowledge soundness.

Definition 3.1 (Completeness). *For all security parameter $\lambda \in \mathbb{N}$ and statement x and witness w such that $(x, w) \in \mathcal{R}$, we have*

$$\Pr \left(b = 1 : \begin{array}{l} \mathbf{pp} \leftarrow \text{Setup}(1^\lambda) \\ b \leftarrow \langle \mathcal{P}(w), \mathcal{V} \rangle(\mathbf{pp}, x) \end{array} \right) = 1.$$

Definition 3.2 (Knowledge Soundness). *For any PPT malicious prover $\mathcal{P}^* = (\mathcal{P}_1^*, \mathcal{P}_2^*)$, there exists a PPT algorithm $\mathcal{E}$ such that the following probability is negligible:*

$$\Pr \left(\begin{array}{l} \mathbf{pp} \leftarrow \text{Setup}(1^\lambda) \\ b = 1 \wedge (x, \mathbf{st}) \leftarrow \mathcal{P}_1^*(1^\lambda, \mathbf{pp}) \\ (x, w) \notin \mathcal{R} : b \leftarrow \langle \mathcal{P}_2^*(\mathbf{st}), \mathcal{V} \rangle(\mathbf{pp}, x) \\ w \leftarrow \mathcal{E}^{\mathcal{P}_2^*}(\mathbf{pp}, x) \end{array} \right).$$

A *succinct* argument of knowledge $\langle \mathcal{P}, \mathcal{V} \rangle$ for a relation $\mathcal{R}$, must satisfy completeness and knowledge soundness and additionally be *succinct*, that is, the communication complexity between prover and verifier, as well as the verification complexity is bounded by $\text{poly}(\lambda, \log |w|)$.

3.2 Polynomial Commitment Scheme

A polynomial commitment scheme (PCS) introduced in [KZG10] allows a prover to open evaluations of the committed polynomial succinctly. A PCS over $\mathbb{F}$ is a tuple $\text{PC} = (\text{Setup}, \text{Com}, \text{Open}, \text{Eval})$ where:

- $\mathbf{pp} \leftarrow \text{Setup}(1^\lambda, n, \{D_i\}_{i \in [n]})$. On input security parameter λ , number of variables n and upper bounds $D_i \in \mathbb{N}$ on the degree of each variable X_i for a n -variate polynomial, **Setup** generates public parameters $\mathbf{pp}$.
- $(C, \tilde{\mathbf{c}}) \leftarrow \text{Com}(\mathbf{pp}, f(\mathbf{X}), \mathbf{d})$. On input the public parameters $\mathbf{pp}$, and a n -variate polynomial $f(X_1, \dots, X_n) \in \mathbb{F}[X_1, \dots, X_n]$ with degree at most $\deg(X_i) = d_i \leq D_i$ for all i , **Com** outputs a commitment to the polynomial C , and additionally an opening hint $\tilde{\mathbf{c}}$.
- $b \leftarrow \text{Open}(\mathbf{pp}, f(\mathbf{X}), \mathbf{d}, C, \tilde{\mathbf{c}})$. On input the public parameters $\mathbf{pp}$, the commitment C and the opening hint $\tilde{\mathbf{c}}$, a polynomial $f(X_1, \dots, X_n)$ with $d_i \leq D_i$, **Open** outputs a bit indicating accept or reject.
- $b \leftarrow \text{Eval}(\mathbf{pp}, C, \mathbf{d}, \mathbf{x}, v; f(\mathbf{X}))$. A public coin interactive protocol

$$\langle P_{\text{eval}}(f(\mathbf{X})), V_{\text{eval}} \rangle(\mathbf{pp}, C, \mathbf{d}, \mathbf{x}, v)$$

between a PPT prover and a PPT verifier. The parties have public parameters $\mathbf{pp}$, commitment C , degree d , evaluation point x , and claimed evaluation v as common input. The prover has, in addition, the opening $f(X_1, \dots, X_n)$ of C , with $\deg(X_i) \leq d_i$. At the end of the protocol, the verifier outputs 1 indicating accepting the proof or outputs 0 indicating rejecting the proof.

A polynomial commitment scheme must satisfy completeness, binding and extractability.

Definition 3.3 (Completeness). *For all polynomials $f(X_1, \dots, X_n) \in \mathbb{F}[X_1, \dots, X_n]$ with degree $\deg(X_i) = d_i \leq D_i$, for all $(x_1, \dots, x_n) \in \mathbb{F}^n$,*

$$\Pr \left[b = 1 \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda, n, \{D_i\}_{i \in [n]}), \\ v \leftarrow f(\mathbf{x}), \\ b \leftarrow \text{Eval}(\text{pp}, C, \mathbf{d}, \mathbf{x}, v; f(\mathbf{X})) \end{array} \right] = 1.$$

Definition 3.4 (Binding). *A polynomial commitment scheme PC is binding if for all PPT $\mathcal{A}$, the following probability is negligible in λ :*

$$\Pr \left(\begin{array}{l} \text{Open}(\text{pp}, f_0, \mathbf{d}_0, C, \tilde{\mathbf{c}}_0) = 1 \wedge \\ \text{Open}(\text{pp}, f_1, \mathbf{d}_1, C, \tilde{\mathbf{c}}_1) = 1 \wedge \\ f_0 \neq f_1 \end{array} \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda, n, \mathbf{D}) \\ (C, f_0, f_1, \tilde{\mathbf{c}}_0, \\ \tilde{\mathbf{c}}_1, \mathbf{d}_0, \mathbf{d}_1) \leftarrow \mathcal{A}(\text{pp}) \end{array} \right).$$

Definition 3.5 (Knowledge Soundness). *For any PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, there exists a PPT algorithm $\mathcal{E}$ such that the following probability is negligible in λ :*

$$\Pr \left(\begin{array}{l} b = 1 \wedge \\ \mathcal{R}_{\text{Eval}}(\text{pp}, C, \mathbf{x}, : \\ v; \tilde{f}, \tilde{\mathbf{c}}) = 0 \end{array} \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda, n, \{D_i\}_{i \in [n]}) \\ (C, \mathbf{d}, \mathbf{x}, v, \text{st}) \leftarrow \mathcal{A}_1(\text{pp}) \\ (\tilde{f}, \tilde{\mathbf{c}}) \leftarrow \mathcal{E}^{\mathcal{A}_2}(\text{pp}, C, d) \\ b \leftarrow \langle \mathcal{A}_2(\text{st}), V_{\text{eval}} \rangle(\text{pp}, C, \mathbf{d}, \mathbf{x}, v) \end{array} \right).$$

where the relation $\mathcal{R}_{\text{Eval}}$ is defined as follows:

$$\mathcal{R}_{\text{Eval}} = \{((\text{pp}, C \in \mathbb{G}, \mathbf{x} \in \mathbb{F}^n, v \in \mathbb{F}); (f(X_1, \dots, X_n), \tilde{\mathbf{c}})) : \\ (\text{Open}(\text{pp}, f, \mathbf{d}, C, \tilde{\mathbf{c}}_0) = 1) \wedge v = f(\mathbf{x})\}$$

Fiat-Shamir. An interactive protocol is *public-coin* if the verifier's messages are uniformly random strings. Public-coin protocols can be transformed into non-interactive arguments in the Random Oracle Model (ROM) by using the Fiat-Shamir (FS) [FS87] heuristic to derive the verifier's messages as the output of a Random Oracle. All protocols in this work are public-coin interactive protocols in the structured reference string (SRS) model where both the parties have access to a SRS, that are then compiled into non-interactive arguments using FS.

3.3 The KZG PCS

The KZG univariate PCS was introduced in [KZG10]. We denote the KZG scheme by the tuple of PPT algorithms (KZG.Setup, KZG.Commit, KZG.Prove, KZG.Verify) as defined below.

Definition 3.6 (KZG PCS). *Let $(\mathbb{F}, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, g_t)$ be output of bilinear group generator $\text{BG}(1^\lambda)$.*

- KZG.Setup on input $(1^\lambda, d)$, where d is the degree bound, outputs

$$\text{srs} = (\{[\tau]_1, \dots, [\tau^d]_1\}, \{[\tau]_2, \dots, [\tau^d]_2\}).$$

- KZG.Commit on input $(\text{srs}, p(X))$, where $p(X) \in \mathbb{F}_{\leq d}[X]$, outputs $C = [p(\tau)]_1$.

- **KZG.Prove** on input $(\text{srs}, p(X), \alpha)$, where $p(X) \in \mathbb{F}_{\leq d}[X]$ and $\alpha \in \mathbb{F}$, outputs (v, π) such that $v = p(\alpha)$ and $\pi = [q(\tau)]_1$, for $q(X) = \frac{p(X) - p(\alpha)}{X - \alpha}$.
- **KZG.Verify** on input $(\text{srs}, C, v, \alpha, \pi)$, outputs 1 if the following equation holds, and 0 otherwise:

$$e(C - v[1]_1 + \alpha\pi, [1]_2) \stackrel{?}{=} e(\pi, [\tau]_2).$$

Definition 3.7 (q-DLOG Assumption). *The q-DLOG assumption with respect to $\mathcal{G}$ holds if for all λ and for all PPT $\mathcal{A}$, the following probability is negligible in λ :*

$$\Pr \left(\begin{array}{l} \tau = \tau' \\ \tau' \leftarrow \mathcal{A}(1^\lambda, \text{pp}) \end{array} : \begin{array}{l} (\mathbb{F}, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, g_t) \leftarrow \text{BG}(1^\lambda), \\ \tau \leftarrow \mathbb{F}, \\ \text{pp} := ([\tau]_1, [\tau^2]_1, \dots, [\tau^q]_1, \\ [\tau]_2, [\tau^2]_2, \dots, [\tau^q]_2) \end{array} \right)$$

KZG is shown to be evaluation binding assuming the hardness of q-DLOG (Definition 3.7) and knowledge sound in the Algebraic Group Model (AGM). At a high level, AGM [FKL18] considers *algebraic* adversaries that are algorithms $\mathcal{A}$ such that every group element output by $\mathcal{A}$ is accompanied by a representation of that group element in terms of all the group elements that $\mathcal{A}$ has seen so far (input and output).

3.4 Polynomial IOP

A modular approach for designing efficient succinct arguments is to (i) first, construct an Interactive Oracle Proof (IOP) that is an information-theoretic protocol in an idealized model, (ii) compile the information-theoretic protocol via a cryptographic compiler to obtain a cryptographic argument system. In a Polynomial IOP (PIOP), the prover provides oracle access to a set of polynomials, and the verifier accepts or rejects by checking certain identities over the polynomials output by the prover and possibly public polynomials known to the verifier. A PIOP is compiled into a succinct argument of knowledge by using a *polynomial commitment scheme* to realize the polynomial oracles. Many recent constructions of zkSNARKs [BFS20, CHM⁺20, GWC19] follow this approach where the information theoretic object is a PIOP and the cryptographic compiler is a polynomial commitment scheme.

Definition 3.8 (Polynomial IOP). *A polynomial IOP is a public-coin interactive proof for a relation $\mathcal{R} = (\mathfrak{x}, \mathfrak{w})$. $\mathcal{R}$ is an oracle relation such that $\mathfrak{x}$ consists of oracles to μ -variate polynomials over $\mathbb{F}$. These oracles can be queried at arbitrary points in $\mathbb{F}^\mu$ to evaluate the polynomial at these points. In every round in the protocol, the prover sends multi-variate polynomial oracles. The verifier in every round sends a random challenge. At the end of the protocol, the verifier (with oracle access to all the polynomial oracles sent so far) and given its own randomness outputs accept/reject. A PIOP satisfies completeness and knowledge-soundness.*

We use the following results about PIOPs and their compilation.

Lemma 3.1 ([CHM⁺20, BFS20, CBBZ23]). *If a PIOP is sound for an oracle relation $\mathcal{R}$ with soundness error δ , then it is knowledge sound for $\mathcal{R}$ with knowledge error δ and the extractor running in time polynomial in the witness size.*

Indexed Relations. An indexed relation $\mathcal{R}$ is a set of triples $(i, \mathfrak{x}, \mathfrak{w})$ where i is the index, $\mathfrak{x}$ is the instance, and $\mathfrak{w}$ is the witness. Typically, an NP relation $((C, \mathfrak{x}); \mathfrak{w})$ given by $C(\mathfrak{x}, \mathfrak{w}) = 1$ for

an arithmetic circuit C is expressed as an indexed relation with index $\mathbf{i}$ encoding the description of the circuit C . A PIOP for indexed relation $\mathcal{R}$ is specified by indexer $\mathcal{I}$, prover $\mathcal{P}$ and verifier $\mathcal{V}$ which operate as follows:

- *Offline Phase:* The indexer $\mathcal{I}$ receives the index $\mathbf{i}$ as input and outputs a set of polynomials encoding $\mathbf{i}$.
- *Online Phase:* The prover $\mathcal{P}(\mathbf{i}, \mathbf{x}, \mathbf{w})$ interacts with the verifier $\mathcal{V}(\llbracket \mathbf{i} \rrbracket, \mathbf{x})$, i.e, the verifier has oracle access to the polynomials output by $\mathcal{I}$ in addition to any oracles in the statement $\mathbf{x}$.

Lemma 3.2 ([CHM⁺20, BFS20, CBBZ23]). *Let $\mathcal{R}$ be a relation over $\mathbb{F}$. Let $PIOP = (\mathcal{I}, \mathcal{P}, \mathcal{V})$ be a PIOP over $\mathbb{F}$ for $\mathcal{R}$ with negligible soundness error, and let*

$$PC = (\text{Setup}, \text{Com}, \text{Open}, \text{Eval})$$

be a polynomial commitment scheme over $\mathbb{F}$ that satisfies completeness, binding and extractability. Then there exists a compiler that compiles the PIOP using PC to obtain a public-coin argument of knowledge $\Pi = (\text{Setup}, \mathcal{P}, \mathcal{V})$ for $\mathcal{R}$. If the PIOP is for an indexed relation, the resulting argument system is a preprocessing argument system.

This succinct argument system with a public-coin verifier is finally transformed into a SNARK via Fiat-Shamir.

3.5 Multivariate Sumcheck

Let $f(X_1, \dots, X_\mu) \in \mathbb{F}[X_1, \dots, X_\mu]$. Consider the claim:

$$\sum_{x_1 \in \{0,1\}} \sum_{x_2 \in \{0,1\}} \cdots \sum_{x_\mu \in \{0,1\}} f(x_1, \dots, x_\mu) = y$$

The above claim takes time $O(|B_\mu|)$ to verify. The sumcheck protocol [LFKN90] allows a verifier $\mathcal{V}$ to outsource this computation to a prover $\mathcal{P}$, where the prover $\mathcal{P}$ sends a number of field elements that is logarithmic in the size of the hypercube, and the verifier $\mathcal{V}$ needs to evaluate f at a single point. We recall the classical sumcheck protocol from [LFKN90] for multilinear polynomials.

- **Common Input:** $\mathcal{V}$ is given oracle access $\llbracket f \rrbracket$ to the multilinear polynomial $\tilde{f} \in \mathbb{F}^{\leq 1}[X_1, \dots, X_\mu]$ and the claimed sum $v = \sum_{\mathbf{x} \in B_\mu} f(\mathbf{x})$. The prover and verifier interact in μ rounds $1, \dots, \mu$ as:
- As its i^{th} message, $\mathcal{P}$ sends univariate polynomial $g_i(X_i)$ such that $g_i(X_i) = \sum_{\mathbf{x}' \in B_{\mu-i}} f(\mathbf{r}, X_i, \mathbf{x}')$, where $\mathbf{r} = (r_1, \dots, r_{i-1})$ is the vector of challenges sent by the verifier in the previous rounds. Note that $\mathbf{r}$ is not defined for $i = 1$, and so we ignore it for $i = 1$.
- $\mathcal{V}$ checks $g_i(1) + g_i(0) = g_{i-1}(r_i)$ for $i > 1$ and $g_1(1) + g_1(0) = v$ for $i = 1$. The verifier aborts if the check fails.
- If $i < \mu$, $\mathcal{V}$ sends $r_i \leftarrow \mathbb{F}$ to the prover.
- For $i = \mu$, $\mathcal{V}$ samples $r_\mu \leftarrow \mathbb{F}$ and checks $g_{\mu-1}(r_\mu) = \tilde{f}(r_1, \dots, r_\mu)$ by querying the oracle $\llbracket f \rrbracket$.

The above protocol has communication complexity $O(\mu)$, with the prover $\mathcal{P}$ computing $O(2^\mu)$ $\mathbb{F}$ -operations.

Multilinear PCS with Constant Proof Size. We recall recent constructions [GPS25, EG25] of multilinear polynomial commitment schemes with constant proof size. The constructions use the following isomorphism between μ -variate multilinear polynomials and univariate polynomials $\mathbb{F}[X]$ of degree $< n$ for $n = 2^\mu$. Define the isomorphism of the $\mathbb{F}$ -vector spaces as:

$$\begin{aligned} \varphi : \mathbb{F}^{\leq 1}[X_1, \dots, X_\mu] &\longrightarrow \mathbb{F}^{< n}[X] \\ \sum_{i=1}^n f_i \tilde{e}_{q_\mu}((X_1, \dots, X_\mu), \langle i \rangle) &\mapsto \sum_{i=1}^n f_i X^{i-1}. \end{aligned}$$

For $\tilde{f} \in \mathbb{F}^{\leq 1}[X_1, \dots, X_\mu]$, we use $\hat{f}$ to denote the univariate polynomial $\varphi(\tilde{f})$. We associate a multilinear polynomial $\tilde{f}$ with the univariate polynomial $\hat{f}$ according to the above isomorphism. Subsequently, we treat the KZG commitment C to the polynomial $\tilde{f}$ as the commitment to the multilinear polynomial $\tilde{f}$. The constructions in [GPS25, EG25] show elegant protocols to prove evaluations $\tilde{f}(\mathbf{z}) = v$ for $\mathbf{z} \in \mathbb{F}^\mu$ and $v \in \mathbb{F}$ with constant proof size and $O_\lambda(n)$ prover cost. Specifically, we will use the Samaritan PCS from [GPS25] which achieves constant proof size of 368 bytes over the BLS12-381 curve.

4 Sumcheck with Sublogarithmic Communication

Let $\tilde{G}(X_1, \dots, X_\ell)$ be a multivariate polynomial of degree d over $\mathbb{F}$. Let $\tilde{f}_1, \dots, \tilde{f}_\ell$ be μ -variate multilinear polynomials. For *improved sumcheck*, we consider a sumcheck instance of the form:

$$\sum_{\mathbf{x} \in B_\mu} \tilde{G}(\tilde{f}_1(\mathbf{x}), \dots, \tilde{f}_\ell(\mathbf{x})) = v. \quad (4)$$

Let $n = 2^\mu$ denote the size of the hypercube. Choose $\rho = \log \log n$, and set $\kappa = \mu - \rho$. Let $m = 2^\kappa$. We observe that $m = n / \log n$. To prove the claim in Equation (4), the prover and verifier initially proceed with the first ρ rounds of the classical multivariate sumcheck protocol described in Section 3.5. Assuming $\mathbf{r} = (r_1, \dots, r_\rho)$ is the set of challenges sent by the verifier so far, the reduced claim is of the form

$$\sum_{\mathbf{x} \in B_\kappa} \tilde{G}(\tilde{f}_1(\mathbf{r}, \mathbf{x}), \dots, \tilde{f}_\ell(\mathbf{r}, \mathbf{x})) = \bar{v}, \quad (5)$$

for some $\bar{v} \in \mathbb{F}$. At this stage, the prover commits to κ -variate multilinear polynomials $\tilde{p}_i, i \in [\ell]$ defined by $\tilde{p}_i(\mathbf{x}) = \tilde{f}_i(\mathbf{r}, \mathbf{x})$ for $\mathbf{x} \in B_\kappa$. Let $\hat{p}_1(X), \dots, \hat{p}_\ell(X)$ denote the corresponding univariate polynomials of degree less than m , such that j^{th} coefficient of $\hat{p}_i$ is $\tilde{p}_i(\langle j \rangle), \forall i \in [\ell], j \in [m]$.

4.1 Using a Univariate PIOP

Our idea is to verify the reduced claim using a univariate PIOP over the much smaller domain of size m . Specifically, we choose the domain to be the set $\mathbb{H}$ consisting of the m roots of unity $\{1, \omega, \dots, \omega^{m-1}\}$. To this end, the prover computes polynomials $\hat{q}_i(X), i \in [\ell]$ such that evaluations of $\hat{q}_i$ over the set $\mathbb{H}$ match the monomial coefficients of polynomial $\hat{p}_i$. Let $\{\mu_k^{\mathbb{H}}(X)\}_{k=1}^m$ denote the

Lagrange basis polynomials for the set $\mathbb{H}$ satisfying $\mu_k^{\mathbb{H}}(\omega^{j-1}) = \delta_{kj} \forall j \in [m]$, where $\delta_{kj} = 1$ if $k = j$, and $\delta_{kj} = 0$ otherwise. We define linear operator ifft on the set $\mathbb{F}^{<m}[X]$ as:

$$\text{ifft} \left(\sum_{i=1}^m a_i X^{i-1} \right) = \sum_{i=1}^m a_i \mu_i^{\mathbb{H}}(X). \quad (6)$$

In terms of the above operator, we have $\hat{q}_i = \text{ifft}(\hat{p}_i)$ for all $i \in [\ell]$. Assuming that the polynomials $\hat{p}_i$ and $\hat{q}_i$ are honest, we can reduce the sumcheck claim in Equation (5) to a univariate sumcheck claim as below:

$$\begin{aligned} \sum_{\mathbf{x} \in B_\kappa} \tilde{G}(\tilde{p}_1(\mathbf{x}), \dots, \tilde{p}_\ell(\mathbf{x})) = \bar{v} &\Leftrightarrow \sum_{i \in [m]} \tilde{G}(\tilde{p}_1(\langle i \rangle), \dots, \tilde{p}_\ell(\langle i \rangle)) = \bar{v} \\ &\Leftrightarrow \sum_{i \in [m]} \tilde{G}(\hat{q}_1(\omega^{i-1}), \dots, \hat{q}_\ell(\omega^{i-1})) = \bar{v}. \end{aligned} \quad (7)$$

The final claim uses the fact that $\hat{q}_i(\omega^{j-1}) = \tilde{p}_i(\langle j \rangle)$ for all $i \in [\ell]$ and $j \in [m]$. Let $\hat{G}(X) = \tilde{G}(\hat{q}_1(X), \dots, \hat{q}_\ell(X))$. Then the final claim is essentially equivalent to the claim $\sum_{i \in [m]} \hat{G}(\omega^{i-1}) = \bar{v}$. Now, we can use the *univariate sumcheck* to establish this claim. As shown in [BCR⁺19], it requires the prover to send polynomial oracles $\hat{g}$ and $\hat{h}$, with $\deg(\hat{h}) \leq m-2$ such that:

$$\hat{G}(X) = (X^m - 1) \cdot \hat{g}(X) + X \cdot \hat{h}(X) + \bar{v}/m. \quad (8)$$

The prover commits to polynomials $\hat{g}(X)$ and $\hat{h}(X)$. To check the identity, the verifier queries the polynomials $\hat{q}_i, i \in [\ell]$, $\hat{g}$, and $\hat{h}$ at a random point and enforces the degree bound on $\hat{h}$. The above polynomials can be computed and committed in $O(\mathcal{C}(\tilde{G}, d, \ell) \cdot m \log m) = O(\mathcal{C}(\tilde{G}, d, \ell) \cdot n)$ $\mathbb{F}$ operations and $O(dm + \ell m) = O((d + \ell)n / \log n)$ $\mathbb{G}_1$ operations, where $\mathcal{C}(\tilde{G}, d, \ell)$ is a constant dependent on the multivariate form $\tilde{G}$, d and ℓ . Note that most $\mathbb{F}$ -operations are incurred in computing polynomial products corresponding to high degree terms in $\tilde{G}$. In a typical application we expect d to be a small constant (e.g., $d = 5$ for HyperPlonk) and thus computing each term of $\tilde{G}$ takes $\approx O(m \log m)$ operations. The entire computation of $\hat{G}$ then takes $O(\|\tilde{G}\| m \log m)$ operations, which is $O(\|\tilde{G}\| n)$ where $\|\tilde{G}\|$ denotes the number of monomial terms in $\tilde{G}$.

In our attempt of proving the equivalence in Relation 7, till now, we manage to retain $O(n)$ time complexity of the prover, as discussed above. Let us now focus on the communication complexity. Naively, apart from commitments to $\hat{g}(X)$ and $\hat{h}(X)$, ℓ more commitments corresponding to $\hat{q}_i(X) \forall i \in [\ell]$ are required to be sent to verifier, thus contributing $(\ell + 2)\mathbb{G}_1$ elements. Further, prover also needs to send one univariate PCS evaluation proof for each of the $(\ell + 2)$ committed polynomials, at a verifier-chosen random point $z \in \mathbb{F}$ to prove the equality of both sides of Equation 8. This contributes $(\ell + 2)$ $\mathbb{G}_1$ elements. Alternatively, prover may choose to send only one evaluation proof for a linear combination of $\hat{q}_i(X)$ at the cost of sending a few evaluations $\hat{q}_i(z)$ in clear. Though all these approach lead to only $O(1)$ communication complexity (excluding the $O(\log \log n)$ communication cost during initial sumcheck rounds), we still care about reducing the concrete communication overhead. This drives us to further optimization in communication cost, as discussed in Section 4.3.

4.2 Checking Consistency of Polynomials

The reduction in Section 4.1 requires that the polynomials $\hat{p}_i$ and $\hat{q}_i$ are correctly computed from the original polynomials f_i and initial challenges $\mathbf{r} = (r_1, \dots, r_\rho)$. In this subsection, we describe how to check this.

Checking Consistency of $\hat{p}_i$. First, we wish to show that $\tilde{p}_i(\cdot) = \tilde{f}_i(\mathbf{r}, \cdot)$ for all $i \in [\ell]$. The verifier uses a challenge γ to batch the checks into an aggregated check:

$$\sum_{i=1}^{\ell} \gamma^{i-1} \tilde{p}_i(\cdot) = \sum_{i=1}^{\ell} \gamma^{i-1} \tilde{f}_i(\mathbf{r}, \cdot).$$

The verifier can homomorphically compute commitments to the polynomials $\tilde{p} = \sum_{i=1}^{\ell} \gamma^{i-1} \tilde{p}_i$ and $\tilde{f} = \sum_{i=1}^{\ell} \gamma^{i-1} \tilde{f}_i$. Thereafter it samples $\mathbf{y} \leftarrow \mathbb{F}^\kappa$ and queries $\tilde{p}$ and $\tilde{f}$ at points $\mathbf{y}$ and $(\mathbf{r}, \mathbf{y})$ respectively. It accepts if the two evaluations are equal. Using the multilinear PCS in [GPS25, EG25], the preceding check incurs $O(m+n)$ cost to the prover and results in constant communication.

Note that in a straightforward approach, the prover needs to send two evaluation proofs corresponding to κ -variate multilinear polynomial $\tilde{p}(\cdot)$ at $\mathbf{y}$ and μ -variate multilinear polynomial $\tilde{f}(\cdot)$ at $(\mathbf{r}, \mathbf{y})$, respectively. However, we reduce it to a single multilinear PCS evaluation proof by (i) extending $\tilde{p}(\cdot)$ into a μ -variate multilinear polynomial $\tilde{p}_{ext}(\cdot)$, and (ii) batching $\tilde{p}_{ext}(\cdot)$ and $\tilde{f}(\cdot)$ as a single μ -variate multilinear polynomial. This reduces the concrete communication cost further. We elaborate the technique in Section 4.3.

Checking Consistency of $\hat{q}_i$. Next, we wish to check that for all $i \in [\ell]$, $\hat{q}_i = \text{ifft}(\hat{p}_i)$. Again, using the linearity of ifft operation, the verifier can batch the checks into a single check. We can use the challenge γ used previously. The verifier can compute the commitment to the polynomials $\hat{q}(X) = \sum_{i=1}^{\ell} \gamma^{i-1} \hat{q}_i(X)$ and $\hat{p}(X) = \sum_{i=1}^{\ell} \gamma^{i-1} \hat{p}_i(X)$. Note that prover has already committed to $\hat{p}(X)$, since it is the same as commitment to its multilinear equivalent $\tilde{p}(\cdot)$ used in the previous consistency check. Prover also has sent the commitments to $\hat{q}_i(X)$ (Section 4.1), and verifier can combine them to compute commitment of $\hat{q}(X)$. A straightforward approach at this point would be the following: check $\hat{q}(X) = \text{ifft}(\hat{p}(X))$ using a dedicated PIOP for *basis translation*. In its natural embodiment, this approach would require 2ℓ commitments to $\hat{p}_i(X)$ and $\hat{q}_i(X)$, $\forall i \in [\ell]$.

A Communication-efficient Alternative. In the next subsection, we demonstrate a more communication-efficient alternative to the above approach where the prover does not need to commit to any of the $\hat{q}_i(X)$ polynomials. In fact, it suffices for the prover to simply send their evaluations at verifier-chosen random point z , i.e., $\hat{q}_i(z)$, $\forall i \in [\ell]$ to verifier. Notably, although the prover sends neither commitments to $\hat{q}_i(X)$ nor evaluation proofs for them at z explicitly, the correctness of these evaluations are implied by the already committed polynomial $\hat{p}(X)$ and the relation $\hat{q}(X) = \text{ifft}(\hat{p}(X)) = \sum_{i=1}^m \hat{p}_i \mu_i^{\mathbb{H}}(X)$. This approach has two-fold benefits:

- Equation 8 in Section 4.1 can be checked using only two commitments corresponding to $\hat{g}(X)$ and $\hat{h}(X)$.
- Instead of devising PIOP for $\hat{q}(X) = \text{ifft}(\hat{p}(X))$, we can prove an equivalent statement $\hat{q}(z) = \sum_{i=1}^m \hat{p}_i \mu_i^{\mathbb{H}}(z)$ for verifier-chosen random point z using a simple inner product argument.

We elaborate on this approach in the next subsection.

4.3 Communication-optimized End-to-End Protocol

We now describe an end-to-end version of our improved sumcheck protocol that puts together the ideas in Sections 4.1 and 4.2 along with some optimizations aimed at reducing the concrete communication cost. Recall that, after the completion of first $\rho = \log \log n$ rounds of classical multivariate sumcheck protocol (Section 3.5), the prover holds ℓ multilinear polynomials of κ variables, denoted with $\tilde{p}_i(\cdot) = \tilde{f}_i(\mathbf{r}, \cdot) \forall i \in [\ell]$, and the verifier has a value $\bar{v} \in \mathbb{F}$. The verifier's challenges for the first ρ rounds are denoted together with $\mathbf{r} \in \mathbb{F}^\rho$. Each $\tilde{p}_i(\cdot)$ is also viewed as $\hat{p}_i(X) = \sum_{j=1}^m \hat{p}_{i,j} X^{j-1}$ such that $\hat{p}_{i,j} = \tilde{p}_i(\langle j \rangle)$. At this point, prover first computes $\hat{q}_i(X) = \text{fft}(\hat{p}_i(X)) \forall i \in [\ell]$. We now prove the reduced claim of Equation 5 in broadly three phases, as detailed below.

Phase 1: Univariate Sumcheck. For simplicity of exposition, assume for now that all of the $\hat{p}_i(X)$ and $\hat{q}_i(X)$ polynomials are honestly computed. Then, by the equivalence relation presented in Equation 7, Equation 5 reduces to the following univariate sumcheck:

$$\sum_{i \in [m]} \hat{G}(\omega^{i-1}) = \bar{v}, \quad \text{where,} \quad \hat{G}(X) = \tilde{G}(\hat{q}_1(X), \dots, \hat{q}_\ell(X)).$$

Going by the univariate sumcheck protocol from [BCR⁺19] and our optimization approach (sending evaluations of $\hat{q}_i(X)$ instead of their commitments and evaluation proofs), the interactive argument is as follows.

- Prover computes $\hat{g}(X)$ and $\hat{h}(X)$, with $\deg(\hat{h}) \leq m-2$ such that $\hat{G}(X) = (X^m - 1)\hat{g}(X) + X\hat{h}(X) + \bar{v}/m$. Next, prover sends commitments to $\hat{g}(X)$ and $\hat{h}(X)$.
- Verifier sends $z \leftarrow \mathbb{F}$.
- Prover computes and sends $\hat{q}_i(z), \forall i \in [\ell]$.
- Verifier computes $\hat{G}(z) = \tilde{G}(\hat{q}_1(z), \dots, \hat{q}_\ell(z))$, and homomorphically combines commitments of $\hat{g}(X)$ and $\hat{h}(X)$ to obtain the commitment of $(z^m - 1)\hat{g}(X) + z\hat{h}(X)$. It then performs univariate PCS verification to verify that $(z^m - 1)\hat{g}(X) + z\hat{h}(X)$ evaluates to $(\hat{G}(z) - \bar{v}/m)$ at $X = z$.

Communication Cost. This phase consists of (i) commitments to $\hat{g}(X)$ and $\hat{h}(X)$, contributing $2\mathbb{G}_1$ elements, (ii) evaluations of all $\hat{q}_i(X)$ at $X = z$, contributing $\ell\mathbb{F}$ elements, (iii) one univariate PCS evaluation proof contributing $1\mathbb{G}_1$ element.

Phase 2: Check Consistency between $\tilde{p}_i(\cdot)$ and $\tilde{f}_i(\mathbf{r}, \cdot) \forall i \in [\ell]$. The idea essentially remains to check that $\tilde{p}(\cdot)$ and $\tilde{f}(\mathbf{r}, \cdot)$ evaluate to the same value at a verifier-chosen random point $\mathbf{y} \in \mathbb{F}^\kappa$, which would naively require two multilinear queries, as discussed in Section 4.2. Our assumption is that verifier is already provided with the commitments to all μ -variate multilinear polynomials $\tilde{f}_i(\cdot), \forall i \in [\ell]$. Now we elaborate on how to restrict the number of multilinear queries to just one.

We define the μ -variate polynomial $\tilde{p}_{ext}$ that vacuously extends $\tilde{p}$ to B_μ as $\tilde{p}_{ext}(X_1, \dots, X_\mu) = \tilde{p}(X_{\rho+1}, \dots, X_\mu)$, i.e., $\tilde{p}_{ext}$ essentially ignores the first ρ variables. Thus, we can obtain the query $\tilde{p}(\mathbf{y})$ as $\tilde{p}_{ext}(\mathbf{r}, \mathbf{y})$. Let $\hat{p}_{ext}(X)$ be the univariate polynomial of degree $< n$ corresponding to μ -variate polynomial $\tilde{p}_{ext}$. We need to relate it to the polynomial $\hat{p}(X)$ of degree at most m corresponding

to $\tilde{p}$. It can be seen that if $(p_1, \dots, p_m)$ are the evaluations of $\tilde{p}$ over B_κ , the evaluations of $\tilde{p}_{ext}$ over B_μ are:

$$\underbrace{p_1, \dots, p_1}_{s \text{ times}}, \underbrace{p_2, \dots, p_2}_{s \text{ times}}, \dots, \underbrace{p_m, \dots, p_m}_{s \text{ times}}$$

where $s = 2^\rho$. This is so because the evaluation of $\tilde{p}_{ext}$ is independent of the first ρ variables. The relation describing the corresponding univariate polynomials $\hat{p}_{ext}(X)$ and $\hat{p}(X)$ is seen to be:

$$\hat{p}_{ext}(X) = \sum_{i=1}^m p_i X^{s(i-1)} (1 + X + \dots + X^{s-1}) = \hat{p}(X^s) \cdot (X^s - 1)/(X - 1) \quad (9)$$

Thus, we require the prover to send the commitment to polynomial $\hat{p}_{ext}$, and the verifier checks the identity in Equation 9. And, we can now batch the multilinear queries for $\tilde{p}_{ext}$ and $\tilde{f}$ into a single query. Following is the interactive argument between prover and verifier.

- Verifier sends $\gamma \leftarrow \mathbb{F}$.
- Prover computes $\hat{p}(X) = \sum_{i=1}^\ell \gamma^{i-1} \hat{p}_i(X)$, and sends commitment to $\hat{p}(X)$. It also computes $\hat{p}_{ext}(X)$ as described above and commits to it. This commitment can also be viewed as the commitment to μ -variate multilinear polynomial $\tilde{p}_{ext}(\cdot)$.
- Verifier sends $\mathbf{y} \leftarrow \mathbb{F}^\kappa$.
- Prover sends multilinear PCS evaluation proof for $\tilde{p}_{ext}(\cdot) - \tilde{f}(\cdot)$ at $(\mathbf{r}, \mathbf{y})$.
- Verifier homomorphically combines the commitments to $\tilde{f}_i(\cdot)$ and $\tilde{p}_{ext}(\cdot)$ to compute commitment to $\tilde{p}_{ext}(\cdot) - \tilde{f}(\cdot)$, and verifies the evaluation proof against the value 0 to check the equality. Then it sends $\eta \leftarrow \mathbb{F}$.
- Prover sends $v_{\text{eta}} = \hat{p}(\eta^s)$ and a univariate evaluation proof for $\hat{p}_{ext}$ at η .
- Verifier checks the evaluation proof against value $v_{\text{eta}} \cdot \frac{\eta^s - 1}{\eta - 1}$.

Communication cost. This phase consists of (i) commitments to $\hat{p}(X)$ and $\hat{p}_{ext}(X)$, contributing $2\mathbb{G}_1$ elements, (ii) the evaluation v_{eta} contributing $1\mathbb{F}$ element, (iii) one univariate PCS proof contributing $1\mathbb{G}_1$ element, and (iv) one multilinear PCS evaluation proof contributing $(6\mathbb{G}_1 + \mathbb{F})$ elements, when instantiated with Samaritan PCS [GPS25].

Phase 3: Check Consistency between $\hat{q}_i(X)$ and $\text{ifft}(\hat{p}_i(X))$, $\forall i \in [\ell]$. In order to prove that for all $i \in [\ell]$, $\hat{q}_i(X) = \text{ifft}(\hat{p}_i(X))$, we batch all of them using the same challenge γ , and the claim reduces to $\hat{q}(X) = \text{ifft}(\hat{p}(X))$, where, $\hat{q}(X) = \sum_{i=1}^\ell \gamma^{i-1} \hat{q}_i(X)$. Recall that the verifier knows $\hat{q}_i(z)$ for all $i \in [\ell]$, which together can be combined to compute $\hat{q}(z)$. Now, the key observation is the following,

$$\hat{q}(z) = \text{ifft}(\hat{p}(z)) = \sum_{j=1}^m p_j \mu_j^{\mathbb{H}}(z),$$

where p_j is the i^{th} coefficient of $\hat{p}$, and $\mu_j^{\mathbb{H}}(z) = \frac{\omega^{j-1}(z^m-1)}{m(z-\omega^{j-1})}$. With this observation, we reduce the claim $\hat{q}(X) = \text{fft}(\hat{p}(X))$ for random verifier challenge z to the following inner product argument,

$$\begin{aligned} \sum_{j=1}^m p_j \cdot \frac{\omega^{j-1}(z^m-1)}{m(z-\omega^{j-1})} &= \hat{q}(z) = \sum_{i=1}^{\ell} \gamma^{i-1} \hat{q}_i(z) = v, \\ \implies \sum_{j=1}^m p_j \cdot \frac{\omega^{j-1}}{z-\omega^{j-1}} &= \frac{mv}{z^m-1} \implies \langle \hat{p}, \hat{A}_z \rangle = \frac{mv}{z^m-1}. \end{aligned}$$

Here, $\hat{A}_z(X) = \sum_{j=1}^m \frac{\omega^{j-1}}{z-\omega^{j-1}} X^{j-1}$, and let us assume that for any $a(X) = \sum_{j=1}^m a_j X^{j-1}$, $b(X) = \sum_{j=1}^m b_j X^{j-1}$, $\langle a, b \rangle$ denotes the value $\sum_{i=1}^m a_i b_i$.

It is interesting to note that the evaluations of $\hat{q}_i(X)$ at z , though not verified explicitly using univariate PCS evaluation proofs, are actually batch-verified here due to the relation between $v = \hat{q}(z)$ and committed polynomial $\hat{p}(X)$, written as $\langle \hat{p}, \hat{A}_z \rangle = \frac{mv}{z^m-1}$. Assuming we have an effective mechanism to verify this inner-product relation, what remains to prove is that $\hat{A}_z(X)$ is correctly constructed.

The polynomial $\hat{A}_z(X)$ does not admit a succinct description due to the presence of $(z-\omega^{j-1})^{-1}$ in its coefficients. However, the “inverse-coefficient” polynomial

$$\hat{B}_z(X) = \sum_{j=1}^m \frac{z-\omega^{j-1}}{\omega^{j-1}} X^{j-1} = z \sum_{i=1}^m (\omega^{-1} X)^{i-1} - \frac{X^m-1}{X-1} = z \Psi_{\omega^{-1}}(X) - \frac{X^m-1}{X-1},$$

where $\Psi_{\omega^{-1}}(X) = \prod_{i=1}^{\kappa} (1 + \omega^{-2^{i-1}} X^{2^{i-1}})$, can be evaluated by the verifier in $O(\kappa) = O(\log m)$ field operations.

The prover commits to $\hat{A}_z$ and proves its correctness via the Hadamard relation $\hat{A}_z \circ \hat{B}_z = \frac{X^m-1}{X-1}$. For $a(X) = \sum_{j=1}^m a_j X^{j-1}$ and $b(X) = \sum_{j=1}^m b_j X^{j-1}$, the Hadamard operator is defined as $a \circ b = c(X) = \sum_{j=1}^m c_j X^{j-1}$, if $c_j = a_j b_j \forall j \in [m]$.

In summary, we need mechanisms to prove two statements: (i) $\langle \hat{p}, \hat{A}_z \rangle = \frac{mv}{z^m-1}$, (ii) $\hat{A}_z \circ \hat{B}_z = \frac{X^m-1}{X-1}$. Here, we state a key technical result from ([EG25], Claim 4.1) in Lemma 4.1 and its variant in Lemma 4.2, which we will use to prove these statements.

Lemma 4.1. *For polynomials $\hat{p} = \sum_{i=1}^m p_i X^{i-1}$ and $\hat{q} = \sum_{i=1}^m q_i X^{i-1}$ in $\mathbb{F}[X]$, let $\langle \hat{p}, \hat{q} \rangle$ denote the sum $\sum_{i=1}^m p_i q_i$. Then, $\langle \hat{p}, \hat{q} \rangle = v$ if and only if there exists a polynomial $\hat{S} \in \mathbb{F}[X]$, computable in $O(m \log m)$ field operations such that:*

$$\hat{p}(X) \cdot \hat{q}(1/X) + \hat{p}(1/X) \cdot \hat{q}(X) = 2v + X \cdot \hat{S}(X) + (1/X) \cdot \hat{S}(1/X)$$

The following variant of Lemma 4.1 will also be useful.

Lemma 4.2. *For polynomials $\hat{p} = \sum_{i=1}^m p_i X^{i-1}$ and $\hat{q} = \sum_{i=1}^m q_i X^{i-1}$, let $\hat{r} = \hat{p} \circ \hat{q}$ be the polynomial $\sum_{i=1}^m r_i X^{i-1}$ with $r_i = p_i q_i$ for all $i \in [m]$. Then, with overwhelming probability over the choice of $\delta \leftarrow \mathbb{F}$, the polynomials $\hat{p}, \hat{q}$ and $\hat{r}$ in $\mathbb{F}^{<m}[X]$ satisfy $\hat{r} = \hat{p} \circ \hat{q}$ if and only if $\langle \hat{p}(\delta X), \hat{q}(X) \rangle = \hat{r}(\delta)$. Equivalently, there exists a polynomial $\hat{S}$, computable in $O(m \log m)$ field operations such that:*

$$\hat{p}(\delta X) \cdot \hat{q}(1/X) + \hat{p}(\delta/X) \cdot \hat{q}(X) = 2\hat{r}(\delta) + X \cdot \hat{S}(X) + (1/X) \cdot \hat{S}(1/X)$$

Proof. The proof essentially follows from batching the checks $r_i = p_i q_i$ using the challenge δ . $\square$

Now, we are sufficiently equipped with techniques to prove $\langle \hat{p}, \hat{A}_z \rangle = \frac{mv}{z^m - 1}$, and $\hat{A}_z \circ \hat{B}_z = \frac{X^m - 1}{X - 1}$. Once prover has committed to $\hat{p}(X)$ and $\hat{A}_z(X)$, by Lemma 4.1, the first relation can be proved by committing to a polynomial $\hat{S}_1(X)$, and providing its univariate PCS evaluation proof at a verifier-chosen random point, where,

$$\hat{p}(X) \cdot \hat{A}_z(1/X) + \hat{p}(1/X) \cdot \hat{A}_z(X) = \frac{2mv}{z^m - 1} + X \cdot \hat{S}_1(X) + (1/X) \cdot \hat{S}_1(1/X).$$

Similarly, by Lemma 4.2, the second relation can be proved by committing to a polynomial $\hat{S}_2(X)$, and providing its univariate PCS evaluation proof at a verifier-chosen random point, where,

$$\hat{A}_z(X) \cdot \hat{B}_z(\delta/X) + \hat{A}_z(1/X) \cdot \hat{B}_z(\delta X) = \frac{2(\delta^m - 1)}{\delta - 1} + X \cdot \hat{S}_2(X) + (1/X) \cdot \hat{S}_2(1/X).$$

We can batch these two evaluation proofs into one for a verifier-chosen random point, say $\epsilon \leftarrow \mathbb{F}$, as following:

$$\begin{aligned} & \hat{p}(X) \cdot \hat{A}_z(1/X) + \hat{p}(1/X) \cdot \hat{A}_z(X) + \epsilon(\hat{A}_z(X) \cdot \hat{B}_z(\delta/X) + \hat{A}_z(1/X) \cdot \hat{B}_z(\delta X)) \\ &= 2\left(\frac{mv}{z^m - 1} + \epsilon \cdot \frac{\delta^m - 1}{\delta - 1}\right) + X \cdot \hat{S}(X) + (1/X) \cdot \hat{S}(1/X). \end{aligned} \quad (10)$$

Now, instead of committing to $\hat{S}_1(X)$ and $\hat{S}_2(X)$, we compute and commit to a single $\hat{S}(X)$, and also provide a univariate PCS evaluation proof to prove the above relation at a random point.

Finally, we summarize this phase in terms of interactive argument between prover and verifier.

- Verifier computes $v = \sum_{i=1}^{\ell} \gamma^{i-1} \hat{q}_i(z)$, with all $\hat{q}_i(z)$ available with it. It sends $\delta, \epsilon \leftarrow \mathbb{F}$.
- Prover computes and commits to $\hat{A}_z(X) = \sum_{j=1}^m \frac{\omega^{j-1}}{z - \omega^{j-1}} X^{j-1}$. It also computes and commits to $\hat{S}(X)$ such that Equation 10 holds.
- Verifier sends $r \leftarrow \mathbb{F}$.
- Prover computes $v_{\text{Ainv}} = \hat{A}_z(1/r)$, $v_{\text{pinv}} = \hat{p}(1/r)$, $v_{\text{Sinv}} = \hat{S}(1/r)$, $v_{\text{Binv}} = \hat{B}_z(\delta/r)$. It sends v_{Ainv} , v_{pinv} , v_{Sinv} to the verifier. Also, it sends univariate PCS evaluation proof of the polynomial $v_{\text{Ainv}} \cdot \hat{p}(X) + (v_{\text{pinv}} + \epsilon \cdot v_{\text{Binv}}) \cdot \hat{A}_z(X) - r \cdot \hat{S}(X)$ at $X = r$.
- Verifier computes $v_{\text{Binv}} = \hat{B}_z(\delta/r)$ and $v_{\text{B}} = \hat{B}_z(\delta r)$ in $O(\log n)$ time. It then homomorphically computes commitment to $v_{\text{Ainv}} \cdot \hat{p}(X) + (v_{\text{pinv}} + \epsilon \cdot v_{\text{Binv}}) \cdot \hat{A}_z(X) - r \cdot \hat{S}(X)$, and verifies the evaluation proof against the value $2\left(\frac{mv}{z^m - 1} + \epsilon \cdot \frac{\delta^m - 1}{\delta - 1}\right) + (1/r) \cdot v_{\text{Sinv}} - \epsilon \cdot v_{\text{Ainv}} \cdot v_{\text{B}}$.

Communication Cost. This phase consists of (i) commitments to $\hat{A}_z(X)$ and $\hat{S}(X)$ contributing $2\mathbb{G}_1$ elements, (ii) v_{Ainv} , v_{pinv} , and v_{Sinv} contributing $3\mathbb{F}$ elements and (iii) one univariate PCS proof contributing $1\mathbb{G}_1$ elements.

Lastly, a degree check is performed combining the degree-check polynomials involved in multilinear PCS instance (Phase 2) as well as $\hat{h}(X)$. Degree check contributes $2\mathbb{G}_1$ elements to communication cost.

Table 2: Comparison of proof sizes for vanilla and improved sumcheck over the BLS12-381 curve using arkworks-0.5.0.

Number of gates (n)	2^{20}	2^{22}	2^{24}	2^{26}	2^{28}	2^{30}
Vanilla sumcheck proof size	1.69KB	1.81KB	1.94KB	2.06KB	2.19KB	2.31KB
Improved sumcheck proof size	1.36KB	1.36KB	1.36KB	1.36KB	1.36KB	1.36KB

4.4 Putting Everything Together

Our proposed improved sumcheck consists of $\rho = \log \log n$ rounds of classical multivariate sumcheck, contributing $\log \log n (\mathbb{G}_1 + \mathbb{F})$ elements, followed by three communication-optimized phases, as described in Section 4.3, which together contributes $O(1)$ elements of $\mathbb{G}_1$ and $\mathbb{F}$. Thus we are able to achieve $O(\log \log n)$ proof size, while retaining $O(n)$ prover time and $O(\log n)$ verifier time. We summarize our improved sumcheck argument in Theorem 4.1.

Theorem 4.1. *Given a multivariate polynomial $\tilde{\mathbf{G}}$ with ℓ variables and total degree d , and μ -variate multilinear polynomials $f_1, \dots, f_\ell$, assuming the security of the KZG univariate PCS [KZG10], there exists an argument of knowledge for the multivariate sumcheck relation*

$$\sum_{\mathbf{x} \in B^\mu} \tilde{\mathbf{G}}(f_1(\mathbf{x}), \dots, f_\ell(\mathbf{x})) = v,$$

where (i) the prover incurs $\mathcal{C}(\tilde{\mathbf{G}}, d, \ell) \cdot n$ $\mathbb{F}$ -operations, $O(n + \ell n / \log n)$ $\mathbb{G}_1$ operations for some constant $\mathcal{C}(\tilde{\mathbf{G}}, d, \ell)$ dependent on the multivariate form $\tilde{\mathbf{G}}$, d and ℓ , (ii) the verifier incurs $O(\ell + \log n)$ $\mathbb{F}$ -operations and $O(\ell)$ $\mathbb{G}_1$ -operations while (iii) the proof size is $O(\ell + d \log \log n)$.

Proof. The proof follows from observing that polynomial IOP for the sumcheck relation in Sections 4.3 is sound, i.e, the verifier accepts with negligible probability if the sumcheck relation is not satisfied. Using KZG [KZG10] to realize the univariate polynomial oracles, and the KZG-based Samaritan PCS from [GPS25] to realize the multilinear polynomial oracles, we obtain a knowledge sound argument using Lemma 3.2. $\square$

Remark. We rely on the security of KZG [KZG10] since it underlies the construction of constant sized multilinear PCS such as Samaritan [GPS25] and Mercury [EG25]. Traditionally, the security of the KZG is argued in the algebraic group model (AGM), which our sumcheck argument would also inherit. However, recently, the security of KZG has been proven from falsifiable assumptions [LPS24], which plausibly hold under weaker models than the AGM. We believe that the multilinear polynomial commitment schemes in [GPS25, EG25] can also plausibly be shown to be secure under these assumptions. We leave a detailed analysis to future work.

Concrete Proof Sizes. In Table 2, we report the concrete proof sizes of our improved multivariate sumcheck argument for the following multivariate sumcheck instance

$$\sum_{\mathbf{x} \in B_\mu} (a(\mathbf{x}) \cdot b(\mathbf{x}) - c(\mathbf{x})) = v$$

We choose the above instance for illustration since it is simple but, at the same time, involves a polynomial multiplication operation. We compare the proof sizes of our improved multivariate

sumcheck argument with the proof sizes of the “vanilla” sumcheck argument from Section 3.5 for $n = 2^\mu$ varying in the range $n = 2^{20}$ to $n = 2^{30}$. The proof sizes are reported over the BLS12-381 curve using arkworks-0.5.0. Notably, at $n = 2^{30}$, the concrete proof size for our improved sumcheck argument is $\approx 1.7\times$ smaller than the vanilla sumcheck argument.

5 HybridSpartan

5.1 Background: MicroSpartan

We recall the MicroSpartan multilinear PIOP [ZSCZ25] for R1CS satisfiability, which is a variant of the earlier Spartan [Set20] PIOP for the same. Let $m, n \in \mathbb{N}$ and let $A, B, C \in \mathbb{F}^{m \times n}$ be sparse matrices with $O(n)$ non-zero entries. Formally, let n_A, n_B and n_C be number of non-zero entries in matrices A, B and C respectively. Let $N = n_A + n_B + n_C$ and let M be the matrix obtained by row-wise concatenation of A, B and C . Let $\{(\text{row}_i, \text{col}_i, \text{val}_i)\}_{i=1}^N$ be a canonical ordering of the non-zero tuples in the sparse representation of M , where we assume that first n_A tuples come from A , next n_B come from B and the remaining n_C come from C . We define vectors $\text{row}, \text{col}, \text{val}_A, \text{val}_B, \text{val}_C \in \mathbb{F}^N$ as follows:

- For $i = 1, \dots, N$: $\text{row}[i] = \text{row}_i, \text{col}[i] = \text{col}_i$.
- For $i = 1, \dots, n_A$: $\text{val}_A[i] = \text{val}_i$.
- For $i = n_A + 1, \dots, n_A + n_B$: $\text{val}_B[i] = \text{val}_i$.
- For $i = n_A + n_B + 1, \dots, N$: $\text{val}_C[i] = \text{val}_i$.

For all of the above vectors, the positions not explicitly specified are assumed to be 0. Let $\widetilde{\text{row}}, \widetilde{\text{col}}, \widetilde{\text{val}}_A, \widetilde{\text{val}}_B$ and $\widetilde{\text{val}}_C$ denote the MLEs of the above vectors. Let $\mathbf{z}$ be R1CS witness satisfying $A\mathbf{z} \circ B\mathbf{z} = C\mathbf{z}$. Henceforth, with suitable zero-padding, we view A, B, C as $N \times N$ matrices and $\mathbf{z} \in \mathbb{F}^N$ satisfying $A\mathbf{z} \circ B\mathbf{z} = C\mathbf{z}$. The prover $\mathcal{P}$ and verifier $\mathcal{V}$ proceed as follows:

1. $\mathcal{P}$ computes $\mathbf{a} = A\mathbf{z}, \mathbf{b} = B\mathbf{z}$ and $\mathbf{c} = C\mathbf{z}$. Let $\tilde{z}, \tilde{a}, \tilde{b}$ and $\tilde{c}$ denote MLEs of vectors $\mathbf{z}, \mathbf{a}, \mathbf{b}$ and $\mathbf{c}$ respectively.
2. $\mathcal{P}$ sends polynomial oracles $[[\tilde{z}]], [[\tilde{a}]], [[\tilde{b}]], [[\tilde{c}]]$.
3. $\mathcal{V}$ sends $\boldsymbol{\tau} \leftarrow \mathbb{F}^{\log N}$.
4. $\mathcal{P}$ computes vectors $\mathbf{L}_r[i] = \mathbf{eq}_r[\text{row}[i]]$ and $\mathbf{L}_c[i] = \mathbf{z}[\text{col}[i]]$. Here $\mathbf{eq}_r$ denotes the vector with i^{th} entry as $\tilde{eq}(\langle i \rangle, \boldsymbol{\tau})$ for $i \in [N]$.
5. $\mathcal{P}$ sends polynomial oracles $[[\tilde{L}_r]], [[\tilde{L}_c]]$.

Next, the following checks are performed on the polynomial oracles:

- *R1CS relation holds*: This amounts to checking that $\tilde{a}(\mathbf{x}) \cdot \tilde{b}(\mathbf{x}) - \tilde{c}(\mathbf{x})$ is 0 for all $\mathbf{x} \in B_{\log N}$. With overwhelming probability, this reduces to the following sumcheck instance:

$$\sum_{\mathbf{x} \in B_{\log N}} \tilde{eq}(\mathbf{x}, \boldsymbol{\tau}) \left(\tilde{a}(\mathbf{x}) \cdot \tilde{b}(\mathbf{x}) - \tilde{c}(\mathbf{x}) \right) = 0 \quad (11)$$

- *Correctness of $\tilde{a}$, $\tilde{b}$, $\tilde{c}$.* To check $\mathbf{a} = A\mathbf{z}$, $\mathbf{b} = B\mathbf{z}$ and $\mathbf{c} = C\mathbf{z}$, $\mathcal{P}$ and $\mathcal{V}$ define following sumcheck instances:

$$\begin{aligned}
\sum_{\mathbf{x} \in B_{\log N}} \left(\tilde{L}_r(\mathbf{x}) \cdot \widetilde{val}_A(\mathbf{x}) \cdot \tilde{L}_c(\mathbf{x}) - \tilde{e}q(\mathbf{x}, \boldsymbol{\tau}) \cdot \tilde{a}(\mathbf{x}) \right) &= 0 \\
\sum_{\mathbf{x} \in B_{\log N}} \left(\tilde{L}_r(\mathbf{x}) \cdot \widetilde{val}_B(\mathbf{x}) \cdot \tilde{L}_c(\mathbf{x}) - \tilde{e}q(\mathbf{x}, \boldsymbol{\tau}) \cdot \tilde{b}(\mathbf{x}) \right) &= 0 \\
\sum_{\mathbf{x} \in B_{\log N}} \left(\tilde{L}_r(\mathbf{x}) \cdot \widetilde{val}_C(\mathbf{x}) \cdot \tilde{L}_c(\mathbf{x}) - \tilde{e}q(\mathbf{x}, \boldsymbol{\tau}) \cdot \tilde{c}(\mathbf{x}) \right) &= 0
\end{aligned} \tag{12}$$

- *Correctness of Lookups:* Finally, to check that vectors $\mathbf{L}_r$ and $\mathbf{L}_c$ correspond to correct lookups on vectors $\mathbf{eq}_{\boldsymbol{\tau}}$ and $\mathbf{z}$ respectively, we follow the “logarithmic derivative” approach of [Hab22]. The prover sends oracles to multiplicity polynomials $\tilde{m}_r$, $\tilde{m}_c$ and shows the equality of the rational functions:

$$\begin{aligned}
\sum_{\mathbf{x} \in B_{\log N}} \frac{\tilde{m}_r(\mathbf{x})}{X + \tilde{id}(\mathbf{x}) \cdot Y + \tilde{e}q(\mathbf{x}, \boldsymbol{\tau})} &= \sum_{\mathbf{x} \in B_{\log N}} \frac{1}{X + \widetilde{row}(\mathbf{x}) \cdot Y + \tilde{L}_r(\mathbf{x})} \\
\sum_{\mathbf{x} \in B_{\log N}} \frac{\tilde{m}_c(\mathbf{x})}{X + \tilde{id}(\mathbf{x}) \cdot Y + \tilde{z}(\mathbf{x})} &= \sum_{\mathbf{x} \in B_{\log N}} \frac{1}{X + \widetilde{col}(\mathbf{x}) \cdot Y + \tilde{L}_c(\mathbf{x})}
\end{aligned} \tag{13}$$

The verifier probabilistically checks the above at $X = \alpha$, $Y = \beta$ for $\alpha, \beta \leftarrow \mathbb{F}$. At this point verifier sends $\gamma \leftarrow \mathbb{F}$, and prover sends $\tilde{g}(\mathbf{x})$ which interpolates the rational function:

$$\begin{aligned}
\tilde{g}(\mathbf{x}) &= \chi_1(\mathbf{x}) + \gamma \cdot \chi_2(\mathbf{x}) \text{ where:} \\
\chi_1(\mathbf{x}) &= \frac{\tilde{m}_r(\mathbf{x})}{\alpha + \beta \cdot \tilde{id}(\mathbf{x}) + \tilde{e}q(\mathbf{x}, \boldsymbol{\tau})} - \frac{1}{\alpha + \beta \cdot \widetilde{row}(\mathbf{x}) + \tilde{L}_r(\mathbf{x})} \\
\chi_2(\mathbf{x}) &= \frac{\tilde{m}_c(\mathbf{x})}{\alpha + \beta \cdot \tilde{id}(\mathbf{x}) + \tilde{z}(\mathbf{x})} - \frac{1}{\alpha + \beta \cdot \widetilde{col}(\mathbf{x}) + \tilde{L}_c(\mathbf{x})}
\end{aligned}$$

The verifier then sends $\boldsymbol{\tau}' \leftarrow \mathbb{F}^{\log N}$ and defines the following sumcheck instances:

$$\sum_{\mathbf{x} \in B_{\log N}} \tilde{e}q(\mathbf{x}, \boldsymbol{\tau}') \cdot (\tilde{D}(\mathbf{x})\tilde{g}(\mathbf{x}) - \tilde{N}(\mathbf{x})) = 0, \quad \sum_{\mathbf{x} \in B_{\log N}} \tilde{g}(\mathbf{x}) = 0 \tag{14}$$

where $\tilde{N}(\mathbf{x})$ and $\tilde{D}(\mathbf{x})$ are such that $\tilde{\chi}_1(\mathbf{x}) + \gamma \cdot \tilde{\chi}_2(\mathbf{x}) = \tilde{N}(\mathbf{x})/\tilde{D}(\mathbf{x})$.

Using a random challenge, all the sumcheck instances in Equations (11), (12) and (14) can be aggregated into a single sumcheck instance of the form

$$\bar{G}(\widetilde{row}, \widetilde{col}, \widetilde{val}_A, \widetilde{val}_B, \widetilde{val}_C, \tilde{z}, \tilde{a}, \tilde{b}, \tilde{c}, \tilde{L}_r, \tilde{L}_c, \tilde{m}_r, \tilde{m}_c, \tilde{g}, \tilde{id}, \tilde{e}q(\cdot, \boldsymbol{\tau}), \tilde{e}q(\cdot, \boldsymbol{\tau}')) = 0 \tag{15}$$

where $\bar{G}$ is a multivariate polynomial of total degree ≤ 6 .

5.2 HybridSpartan via Improved Sumcheck

We compile the aforementioned PIOP into a SNARK as follows: (i) we plug in our improved sumcheck argument from Theorem 4.1 for the sumcheck instance 15 (which incurs $O(\log \log n)$ proof size and $O(n)$ prover work), (ii) we realize the multilinear polynomial oracles using Samaritan PCS [GPS25] (which also relies on the univariate KZG PCS [KZG10], but achieves $O(1)$ proof size). We call the resulting (Fiat-Shamir transformed) SNARK **HybridSpartan**. Concretely, we state the following theorem.

Theorem 5.1 (HybridSpartan, Theorem 2.2 restated). *Compiling the MicroSpartan multilinear PIOP in Section 5.1 using our sumcheck argument from Theorem 4.1 and Samaritan PCS [GPS25] yields a SNARK that allows proving the satisfiability of $O(n)$ R1CS constraints such that: (i) the proof size is $O(\log \log n)$, (ii) the prover effort is $O(n)$, and (iii) the verifier effort is $O(\log n)$, while satisfying completeness and knowledge-soundness in the ROM, assuming the security of the KZG univariate PCS [KZG10].*

5.3 Concrete Proof Size of HybridSpartan

In this subsection, we present a detailed analysis of the concrete proof size of **HybridSpartan**. Since the MicroSpartan PIOP boils down to an instance of multivariate sumcheck protocol, with 17 multilinear polynomials, as given in Equation 15, and the maximum degree of a variable in the multivariate sumcheck instance is 6, following is the elaborated proof size analysis of **HybridSpartan**:

- **HybridSpartan** prover sends commitments to the following multilinear polynomials: $\tilde{z}, \tilde{a}, \tilde{b}, \tilde{c}, \tilde{L}_r, \tilde{L}_c, \tilde{m}_r, \tilde{m}_c, \tilde{g}$. The other ones are either committed during a preprocessing phase (e.g., $\tilde{row}, \tilde{col}, \tilde{val}_A, \tilde{val}_B, \tilde{val}_C, \tilde{id}$) or their commits are not computed, since those multilinear polynomials (e.g., $\tilde{eq}(\cdot, \tau), \tilde{eq}(\cdot, \tau')$) can be efficiently evaluated in verifier end. So, **HybridSpartan** proof size is $9\mathbb{G}_1 + |\pi_{\text{sumcheck}}|$.
- The improved sumcheck prover contributes $6 \log \log n \mathbb{F}$ in initial sumcheck phase, $3\mathbb{G}_1 + 17\mathbb{F}$ in Phase 1, $3\mathbb{G}_1 + 1\mathbb{F} + |\pi_{\text{mlpcs}}|$ in Phase 2, $3\mathbb{G}_1 + 3\mathbb{F}$ in Phase 3, and $2\mathbb{G}_1$ for degree check (Section 4.3).
- When we instantiate the improved sumcheck with Samaritan PCS [GPS25], we get $|\pi_{\text{mlpcs}}| = (6\mathbb{G}_1 + \mathbb{F})$.

Summing up, the proof size is $6 \log \log n \mathbb{F} + 26\mathbb{G}_1 + 22\mathbb{F}$. Using the optimization from [CBBZ23, Section 3.1], we further reduce the proof size to $\log \log n (1\mathbb{G}_1 + 1\mathbb{F}) + 26\mathbb{G}_1 + 22\mathbb{F}$. For $n = 2^{30}$, **HybridSpartan**'s proof size is:

- 1.81KB over the BN254 curve, where $|\mathbb{G}_1| = 32$ bytes and $|\mathbb{F}| = 32$ bytes.
- 2.3KB over the BLS12-381 curve, where $|\mathbb{G}_1| = 48$ bytes and $|\mathbb{F}| = 32$ bytes.

In particular, for $n = 2^{30}$, **HybridSpartan**'s proof size over BN254 is $\approx 3.8\times$ smaller than the proof size of Microspartan [ZSCZ25] over BN254.

6 HybridPlonk

6.1 Background: HyperPlonk

HyperPlonk [CBBZ23] is a multilinear PIOP which is an adaptation of Plonk [GWC19], a popular univariate PIOP. HyperPlonk leverages sumcheck protocol to achieve efficient prover compared to Plonk. We describe the variant of HyperPlonk for gates with 3 wires, and 5 selectors. Let $\mu \in \mathbb{N}$ and let $n = 2^\mu$ be the number of gates. We specify the HyperPlonk circuit $\mathcal{C}$ as below:

- Vectors $\mathbf{q}_M, \mathbf{q}_L, \mathbf{q}_R, \mathbf{q}_O$ and $\mathbf{q}_C$ in $\mathbb{F}^n$ denote the selectors.
- Permutation $\sigma : [3n] \rightarrow [3n]$ denotes the wiring constraints.
- The vector $\mathbf{z} = (z_1, \dots, z_{3n})$ denotes the wires of the circuit. Without loss of generality, we assume $z_1, \dots, z_k$ denote the k public input wires.

We say that $\mathbf{z} \in \mathbb{F}^{3n}$ satisfies $(\mathcal{C}, \mathbf{x})$ for $\mathbf{x} \in \mathbb{F}^k$ if it satisfies:

1. **Input Constraints:** $z_i = x_i$ for $i \in [k]$.
2. **Gate Constraints:** $\mathbf{q}_{M,i} \cdot z_i \cdot z_{i+n} + \mathbf{q}_{L,i} \cdot z_i + \mathbf{q}_{R,i} \cdot z_{i+n} + \mathbf{q}_{O,i} \cdot z_{i+2n} + \mathbf{q}_{C,i} = 0$ for all $i \in [n]$.
3. **Wiring Constraints:** $z_{\sigma(i)} = z_i$ for $i \in [3n]$.

The HyperPlonk indexer outputs the following multilinear polynomials corresponding to the circuit $\mathcal{C}$:

- For each $X \in \{M, L, R, O, C\}$, the multilinear polynomial $\tilde{q}_X$ satisfying $\tilde{q}_X(\langle i \rangle) = \mathbf{q}_{X,i}$ for $i \in [n]$.
- Multilinear polynomials $\tilde{\sigma}_j$ satisfying $\tilde{\sigma}_j(\langle i \rangle) = \sigma((j-1)n + i)$ for $j \in [3]$.
- Multilinear polynomials $\tilde{\text{id}}_j$, $j \in [3]$ for the identity permutation satisfying $\tilde{\text{id}}_j(\langle i \rangle) = i + (j-1)n$.

We note that the polynomials for identity permutation can be computed by the verifier in $O(\log n)$ $\mathbb{F}$ -operations, so indexer need not output them.

6.1.1 HyperPlonk PIOP.

We now describe the online phase of the HyperPlonk PIOP [CBBZ23] between a prover $\mathcal{P}$ and a verifier $\mathcal{V}$. To show knowledge of $\mathbf{z} \in \mathbb{F}^{3n}$ satisfying $(\mathcal{C}, \mathbf{x})$ the prover $\mathcal{P}$ encodes the witness as multilinear polynomials $\tilde{w}_1, \tilde{w}_2$ and $\tilde{w}_3$ satisfying $\tilde{w}_1(\langle i \rangle) = z_i$, $\tilde{w}_2(\langle i \rangle) = z_{i+n}$, $\tilde{w}_3(\langle i \rangle) = z_{i+2n}$. As a technical point, $\mathcal{P}$ sends z_i as 0 for $i \in [k]$ to accommodate public inputs. It then sends oracles $\llbracket \tilde{w}_1 \rrbracket$, $\llbracket \tilde{w}_2 \rrbracket$ and $\llbracket \tilde{w}_3 \rrbracket$. The verifier $\mathcal{V}$ computes the multilinear polynomial $\tilde{I}$ for the public inputs satisfying $\tilde{I}(\langle i \rangle) = x_i$ for $i \in [k]$ and 0 elsewhere. It adjusts the oracle $\llbracket \tilde{w}_1 \rrbracket$ sent by $\mathcal{V}$ to $\llbracket \tilde{w}_1 \rrbracket + \llbracket \tilde{I} \rrbracket$. Henceforth, we will use $\llbracket \tilde{w}_1 \rrbracket$ to denote this “adjusted” oracle. The PIOP proceeds as follows:

- $\mathcal{P}$ sends the witness oracles $\llbracket \tilde{w}_i \rrbracket$, $i \in [3]$ as above.
- $\mathcal{V}$ sends challenges $\beta, \gamma \leftarrow \mathbb{F}$.

- On receiving the challenges, $\mathcal{P}$ computes $\mu + 1$ variate oracle $\tilde{v}$ satisfying the following for all $\mathbf{x} \in B_\mu$.

$$\tilde{v}(0, \mathbf{x}) = \prod_{i=1}^3 \left(\frac{\tilde{w}_i(\mathbf{x}) + \beta \cdot \tilde{\text{id}}_i(\mathbf{x}) + \gamma}{\tilde{w}_i(\mathbf{x}) + \beta \cdot \tilde{\sigma}_i(\mathbf{x}) + \gamma} \right), \quad \tilde{v}(1, \mathbf{x}) = \tilde{v}(\mathbf{x}, 0) \cdot \tilde{v}(\mathbf{x}, 1).$$

- $\mathcal{P}$ computes μ -variate polynomials $\tilde{v}_0$ and $\tilde{v}_1$ where $\tilde{v}_b(\mathbf{X}) = \tilde{v}(b, \mathbf{X})$ for $b \in \{0, 1\}$. $\mathcal{P}$ also computes μ -variate polynomials $\tilde{u}_0$ and $\tilde{u}_1$ where $\tilde{u}_b(\mathbf{X}) = \tilde{v}(\mathbf{X}, b)$ for $b \in \{0, 1\}$.
- $\mathcal{V}$ sends batching challenge $\xi \leftarrow \mathbb{F}$ and a zero-check challenge $\tau \leftarrow \mathbb{F}^\mu$.
- $\mathcal{P}$ and $\mathcal{V}$ execute the sumcheck PIOP:

$$\sum_{\mathbf{x} \in B_\mu} \tilde{e}q_\mu(\mathbf{x}, \tau) \cdot \tilde{G}(\mathbf{x}) = 0,$$

where $\tilde{G}$ is defined as:

$$\begin{aligned} \tilde{G}(\mathbf{x}) &= \tilde{q}_M(\mathbf{x})\tilde{w}_1(\mathbf{x})\tilde{w}_2(\mathbf{x}) + \tilde{q}_L(\mathbf{x})\tilde{w}_1(\mathbf{x}) + \tilde{q}_R(\mathbf{x})\tilde{w}_2(\mathbf{x}) + \tilde{q}_O(\mathbf{x})\tilde{w}_3(\mathbf{x}) + \tilde{q}_C(\mathbf{x}) \\ &\quad + \xi \cdot \tilde{v}_0(\mathbf{x}) \prod_{i=1}^3 (\tilde{w}_i(\mathbf{x}) + \beta \tilde{\sigma}_i(\mathbf{x}) + \gamma) - \xi \cdot \prod_{i=1}^3 (\tilde{w}_i(\mathbf{x}) + \beta \tilde{\text{id}}_i(\mathbf{x}) + \gamma) \\ &\quad + \xi^2 \cdot (\tilde{v}_1(\mathbf{x}) - \tilde{u}_0(\mathbf{x}) \cdot \tilde{u}_1(\mathbf{x})). \end{aligned} \tag{16}$$

As noted in [CBBZ23], the oracles $\llbracket \tilde{u}_b \rrbracket$ for $b \in \{0, 1\}$ can be simulated using the oracles $\llbracket \tilde{v}_0 \rrbracket$ and $\llbracket \tilde{v}_1 \rrbracket$. We will however require the prover to explicitly commit to oracles $\llbracket \tilde{u}_b \rrbracket$ instead of simulating them as it helps us to reduce the number of multilinear oracle queries.

6.2 HybridPlonk via Improved Sumcheck

In this subsection, we show how to compile an adaptation of the HyperPlonk multilinear PIOP [CBBZ23] with our improved sumcheck argument and Samaritan PCS, along with some additional univariate checks (described below). The resulting (Fiat-Shamir transformed) SNARK, which we call HybridPlonk, is captured by the following theorem.

Theorem 6.1 (HybridPlonk, Theorem 2.3 restated). *Compiling an adaptation of the HyperPlonk PIOP using Samaritan PCS [GPS25] yields a SNARK that satisfies completeness and knowledge-soundness in the ROM, and allows proving the satisfiability of $O(n)$ Plonkish constraints such that: (i) the proof size is $O(\log \log n)$, (ii) the prover effort is $O(n)$, and (iii) the verifier effort is $O(\log n)$, while satisfying completeness and knowledge-soundness in the ROM, assuming the security of the KZG univariate PCS [KZG10].*

We now present a detailed description of HybridPlonk. As described in Section 6.1, the HyperPlonk PIOP consists of: (i) one sumcheck instance represented by $\sum_{\mathbf{x} \in B_\mu} \tilde{e}q_\mu(\mathbf{x}, \tau) \cdot \tilde{G}(\mathbf{x}) = 0$, and (ii) a check for correct construction of $\tilde{u}_b$ and $\tilde{v}_b$ for $b \in \{0, 1\}$. The sumcheck instance is a function of the following set of multilinear polynomials,

$$(\tilde{e}q(\cdot, \tau), \tilde{\text{id}}_1, \tilde{\text{id}}_2, \tilde{\text{id}}_3, \tilde{q}_M, \tilde{q}_L, \tilde{q}_R, \tilde{q}_O, \tilde{q}_C, \tilde{\sigma}_1, \tilde{\sigma}_2, \tilde{\sigma}_3, \tilde{w}_1, \tilde{w}_2, \tilde{w}_3, \tilde{v}_0, \tilde{v}_1, \tilde{u}_0, \tilde{u}_1),$$

which we prove by directly invoking the improved sumcheck protocol from Section 4. Note that the sumcheck instance has 19 multilinear polynomials, and the combined multivariate polynomial has degree ≤ 5 .

Next, we elaborate on how to prove correct construction of $\tilde{u}_0, \tilde{u}_1, \tilde{v}_0, \tilde{v}_1$, by formulating a suitable relation among them and then prove it using one univariate PCS query. Note that $\tilde{u}_0$ and $\tilde{u}_1$, if constructed correctly must satisfy,

$$\tilde{u}_b(X_1, \dots, X_\mu) = (1 - X_1)\tilde{v}_0(X_2, \dots, X_\mu, b) + X_1\tilde{v}_1(X_2, \dots, X_\mu, b). \quad (17)$$

To prove the above, we define $(\mu + 1)$ -variate polynomial:

$$\tilde{v}(X_1, \dots, X_{\mu+1}) = (1 - X_1)\tilde{v}_0(X_2, \dots, X_{\mu+1}) + X_1\tilde{v}_1(X_2, \dots, X_{\mu+1}). \quad (18)$$

Then we have $\tilde{v}_b(\mathbf{X}) = \tilde{v}(b, \mathbf{X})$ and $\tilde{u}_b(\mathbf{X}) = \tilde{v}(\mathbf{X}, b)$ for $b \in \{0, 1\}$. Interpreting the preceding relations combinatorially, the $2n$ coefficients of $\tilde{v}$ contain the n coefficients of $\tilde{v}_0$ at *even* positions and the n coefficients of $\tilde{v}_1$ at *odd* positions. In terms of their corresponding univariate polynomials, this is equivalent to $\hat{v}(X) = \hat{v}_0(X^2) + X \cdot \hat{v}_1(X^2)$.

We observe that since $\tilde{u}_b$ is determined by restricting the most significant bit, coefficients of $\tilde{u}_0$ constitute the first n coefficients of $\tilde{v}$, while coefficients of $\tilde{u}_1$ constitute the latter n coefficients of $\tilde{v}$. In terms of univariate polynomials, this is equivalent to $\hat{v}(X) = \hat{u}_0(X) + X^n \cdot \hat{u}_1(X)$. Equating, the correctness of polynomials $\tilde{u}_b$, $b \in \{0, 1\}$ is checked using the univariate identity:

$$\hat{u}_0(X) + X^n \cdot \hat{u}_1(X) = \hat{v}_0(X^2) + X \cdot \hat{v}_1(X^2). \quad (19)$$

This identity is verified using one univariate PCS query at a verifier-chosen random point, which additionally contributes $(1\mathbb{G}_1 + 1\mathbb{F})$ elements.

6.3 Concrete Proof Size of HybridPlonk

In this subsection, we present a detailed analysis of the concrete proof size of HybridPlonk. Note that the HybridPlonk prover sends commitments to seven multilinear polynomials: $\tilde{w}_1, \tilde{w}_2, \tilde{w}_3, \tilde{u}_0, \tilde{u}_1, \tilde{v}_0$, and $\tilde{v}_1$. The remaining polynomials are known publicly; hence, the prover need not commit to them. Overall, HybridPlonk's proof size is $7\mathbb{G}_1 + |\pi_{\text{sumcheck}}| + (1\mathbb{G}_1 + 1\mathbb{F})$, where the cost of $(1\mathbb{G}_1 + 1\mathbb{F})$ is incurred by the identity check involving $\tilde{u}_b$ and $\tilde{v}_b$ for $b \in \{0, 1\}$. Following the same analysis of $|\pi_{\text{sumcheck}}|$ from Section 5.3, we conclude that proof size of HybridPlonk is $\log \log n(1\mathbb{G}_1 + 1\mathbb{F}) + 25\mathbb{G}_1 + 25\mathbb{F}$. For $n = 2^{30}$, HybridPlonk's proof size is:

- 2.34KB over the BLS12-381 curve, where $|\mathbb{G}_1| = 48$ bytes and $|\mathbb{F}| = 32$ bytes.
- 1.88KB over the BN254 curve, where $|\mathbb{G}_1| = 32$ bytes and $|\mathbb{F}| = 32$ bytes.

In particular, for $n = 2^{30}$, HybridPlonk's proof size over BLS12-381 is $\approx 2.5\times$ smaller than the proof size of HyperPlonk [ZSCZ25] compiled with Samaritan PCS [GPS25] over the BLS12-381 curve.

7 Implementation and Evaluation

In this section, we evaluate our implementations of HybridSpartan and HybridPlonk, and compare them with three state-of-the-art prover-efficient SNARKs, namely HyperPlonk [CBBZ23] compiled with Samaritan PCS [GPS25] (abbreviated as HyperPlonk + Samaritan PCS from here on),

Table 3: Comparison of proof sizes for various SNARKs. Plonk [GWC19] is not prover-efficient and is only included for reference.

Number of gates (n)	2^{20}	2^{22}	2^{24}	2^{26}	2^{28}	2^{30}
Plonk [GWC19] (BLS12-381)	0.62 KB	0.62 KB	0.62 KB	0.62 KB	0.62 KB	0.62 KB
Spartan [Set20] (BLS12-381)	234.36 KB	394.58 KB	703.1KB	> 1 MB	> 1 MB	> 1 MB
HyperPlonk [CBBZ23] + Samaritan PCS [GPS25] (BLS12-381)	4.22 KB	4.54 KB	4.86 KB	5.18KB	5.5KB	5.82KB
MicroSpartan [ZSCZ25] (BN254)	4.96 KB	5.34 KB	5.73 KB	6.11 KB	6.5 KB	6.88 KB
HybridSpartan (this work, BN254)	1.81 KB	1.81 KB	1.81 KB	1.81 KB	1.81 KB	1.81 KB
HybridPlonk (this work, BLS12-381)	2.34 KB	2.34 KB	2.34 KB	2.34 KB	2.34 KB	2.34 KB

Spartan [Set20] and MicroSpartan [ZSCZ25]. In particular, we point out that HyperPlonk + Samaritan PCS and MicroSpartan are the two prover-efficient SNARKs with the smallest proof sizes currently. Our artifacts are available at: <https://github.com/SayaniSinha97/HybridSNARK>.

Implementation Details. We benchmark our implementations of HybridPlonk and HyperPlonk compiled with Samaritan PCS over the BLS12-381 curve (with $|\mathbb{G}_1| = 48$ bytes and $|\mathbb{F}| = 32$ bytes) using arkworks-0.5.0. For Spartan, we report proof sizes and prover times using a publicly available implementation from <https://github.com/arkworks-rs/spartan/tree/e0aff4c> over BLS12-381 using arkworks-0.5.0. For MicroSpartan, we report proof sizes and prover times using the implementation available at <https://github.com/microsoft/Nova/tree/148a2d4>, which is specifically optimized to use halo2curves-0.9.0 over the BN254 curve (with $|\mathbb{G}_1| = |\mathbb{F}| = 32$ bytes). For a fair comparison with MicroSpartan, we also benchmark HybridSpartan over BN254. However, our implementation of HybridSpartan supports a flexible choice of curves, and can be easily benchmarked over BLS12-381 as well.

Platform. All benchmarks reported in this section were run on an Intel(R) Xeon(R) Silver 4214R CPU with 2.40GHz of clock frequency, 48 cores, and 128GB RAM, running Ubuntu 22.04. To highlight protocol-level comparison, all benchmarks are reported for single-threaded implementation.⁶ We leave comparison studies with parallelized implementations to future work.

Proof Size. Table 3 compares the proof sizes of the above prover-efficient SNARKs when the number of gates/constraints n varies in the range $n = 2^{20}$ to $n = 2^{30}$. We also report the proof sizes for Plonk [GWC19] (which is not prover-efficient but has very small proof sizes) for reference.

We highlight that the proof sizes of HybridSpartan over BN254 and HybridPlonk over BLS12-381 are 1.81KB and 2.34 KB, respectively, which remain effectively unchanged across the range $n = 2^{20}$ to $n = 2^{30}$. This demonstrates that an asymptotic improvement of the proof size from $O(\log n)$ (as was the case for state-of-the-art prover-efficient SNARKs) to $O(\log \log n)$ in HybridSpartan and HybridPlonk has significant impact on the concrete proof size at instance sizes of practical interest. In particular, at circuit sizes of 2^{30} , we observe the following:

- The concrete proof sizes of HybridSpartan and HybridPlonk are at least two orders of magnitude smaller than that of Spartan.
- Over BN254, the proof size of HybridSpartan is $\approx 3.8\times$ smaller than that of MicroSpartan.

⁶We constrain all benchmarks to use a single CPU core.

Table 4: Comparison of prover times for prover-efficient SNARKs.

Number of gates (n)	2^{18}	2^{20}	2^{22}	2^{24}	2^{26}
Spartan [Set20] (BLS12-381)	63.94s	220.73s	801.72s	3023.85s	—
HyperPlonk [CBBZ23] + Samaritan PCS [GPS25] (BLS12-381)	39.34s	144.01s	534.21s	2230.89s	9035.19s
MicroSpartan [ZSCZ25] (BN254)	93s	337.99s	1320.59s	5776.75s	—
HybridSpartan (this work, BN254)	33.29s	114.36s	446.75s	2021.60s	10108.00s
HybridPlonk (this work, BLS12-381)	45.61s	165.47s	618.92s	2535.32s	10341.00s

- Over BLS12-381, the proof size of HybridPlonk is $\approx 2.5\times$ smaller than HyperPlonk compiled with Samaritan PCS.

This establishes HybridSpartan and HybridPlonk as the most compact prover-efficient SNARKs for R1CS and Plonkish constraints, respectively.

Prover Time. Table 4 compares the prover times for the above prover-efficient SNARKs when the number of gates/constraints n varies in the range $n = 2^{18}$ to $n = 2^{26}$. We could not run single-threaded provers for these schemes with larger values of n on the aforementioned platform. Both Spartan[Set20] and MicroSpartan [ZSCZ25] did not scale to $n = 2^{26}$.

MicroSpartan vs HybridSpartan. The prover for HybridSpartan is $\approx 2.8 - 2.9\times$ faster than that of MicroSpartan across all values of n , despite the additional prover cost incurred by our improved sumcheck argument over the vanilla sumcheck argument. This is because our implementation of HybridSpartan commits to a smaller number of multilinear polynomials than the publicly available implementation of MicroSpartan that we benchmark. In particular, the implementation of MicroSpartan uses a larger number of multilinear queries to retain a lower sumcheck degree (≤ 2). On the other hand, our implementation of HybridSpartan directly checks the identities in Equation 13 (which involves four rational functions) using only two sumcheck instances in Equation 14. While this requires a batched sumcheck instance of higher degree (≤ 6), the significantly smaller number of rounds ($O(\log \log n)$) in our improved sumcheck argument compensates for the higher degree of the sumcheck instance, resulting in significantly more efficient proving overall for HybridSpartan.

HyperPlonk vs HybridPlonk. The prover for HybridPlonk is $\approx 14\%$ slower than that of HyperPlonk + Samaritan PCS across all values of n . The additional prover cost for HybridPlonk corresponds to the additional prover cost incurred by our improved sumcheck argument over the vanilla sumcheck argument.

Verifier Time. Spartan’s verifier is $O(\log^2 n)$, while all of HyperPlonk + Samaritan PCS, MicroSpartan, HybridSpartan, and HybridPlonk feature $O(\log n)$ verifier. Hence, the concrete verification times for Spartan are an order of magnitude larger than that of these other SNARKs. Concretely, for $n = 2^{20}$, the verification times for Spartan, HyperPlonk + Samaritan PCS, MicroSpartan, HybridSpartan, and HybridPlonk are approximately 850ms, 11ms, 8ms, 18ms, and 16ms respectively (again, for MicroSpartan and HybridSpartan, the numbers are reported over the BN254 curve, while the rest are reported over BLS12-381).

Note that, for HyperPlonk + Samaritan PCS, MicroSpartan, HybridSpartan, and HybridPlonk, the verification times remain relatively unchanged for larger values of n , since the pairing check

dominates the verification time for all of them. Hence, we do not explicitly report verifier times for different values of n .

Finally, we remark that MicroSpartan’s small verification time can be partly attributed to specific implementation-level optimizations associated with the `halo2curves` library used in the public implementation that we benchmark. We leave it as an interesting future work to achieve similarly optimized implementations of HybridSpartan and HybridPlonk.

Summary. Our benchmarking results demonstrate that, at instance sizes of practical interest, HybridSpartan and HybridPlonk achieve significantly smaller proof sizes than the most compact prover-efficient state-of-the-art SNARKs for R1CS and Plonkish constraints, respectively, while retaining comparable prover costs (in the case of HybridPlonk) or achieving much smaller prover costs (in the case of HybridSpartan), and while incurring moderate verification overheads.

8 Extensions and Future Work

We leave it as an interesting open question to adapt our improved sumcheck argument to achieve sublogarithmic proof sizes for other SNARKs such as Gemini [BCHO22]. Our improved multivariate sumcheck argument is of independent interest, and could have applications beyond SNARKs, such as in lookup arguments [STW24]. We leave such explorations as interesting future work.

References

- [ac22] arkworks contributors. `arkworks` zksnark ecosystem, 2022.
- [ACFY24] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. STIR: Reed-solomon proximity testing with fewer queries. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part X*, volume 14929 of *LNCS*, pages 380–413. Springer, Cham, August 2024.
- [ACFY25] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. WHIR: Reed-solomon proximity testing with super-fast verification. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part IV*, volume 15604 of *LNCS*, pages 214–243. Springer, Cham, May 2025.
- [ARZR25] Noor Athamnah, Noga Ron-Zewi, and Ron D. Rothblum. Linear prover IOPs in log star rounds. Cryptology ePrint Archive, Paper 2025/1269, 2025.
- [BBHR18] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast reed-solomon interactive oracle proofs of proximity. In Ioannis Chatzigiannakis, Christos Kaklamanis, Dániel Marx, and Donald Sannella, editors, *ICALP 2018*, volume 107 of *LIPIcs*, pages 14:1–14:17. Schloss Dagstuhl, July 2018.
- [BCCT12] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. From extractable collision resistance to succinct non-interactive arguments of knowledge, and back again. In *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*, pages 326–349, 2012.

- [BCF⁺25a] Anubhav Baweja, Alessandro Chiesa, Elisabetta Fedele, Giacomo Fenzi, Pratyush Mishra, Tushar Mopuri, and Andrew Zitek-Estrada. Time-space trade-offs for sum-check. Cryptology ePrint Archive, Paper 2025/1473, 2025.
- [BCF⁺25b] Martijn Brehm, Binyi Chen, Ben Fisch, Nicolas Resch, Ron D. Rothblum, and Hadas Zeilberger. Blaze: Fast SNARKs from interleaved RAA codes. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part IV*, volume 15604 of *LNCS*, pages 123–152. Springer, Cham, May 2025.
- [BCG⁺13] Eli Ben-Sasson, Alessandro Chiesa, Daniel Genkin, Eran Tromer, and Madars Virza. SNARKs for C: Verifying program executions succinctly and in zero knowledge. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part II*, volume 8043 of *LNCS*, pages 90–108. Springer, Berlin, Heidelberg, August 2013.
- [BCG⁺14] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. Zerocash: Decentralized anonymous payments from bitcoin. In *2014 IEEE Symposium on Security and Privacy*, pages 459–474. IEEE Computer Society Press, May 2014.
- [BCHO22] Jonathan Bootle, Alessandro Chiesa, Yuncong Hu, and Michele Orrù. Gemini: Elastic SNARKs for diverse environments. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part II*, volume 13276 of *LNCS*, pages 427–457. Springer, Cham, May / June 2022.
- [BCI⁺13] Nir Bitansky, Alessandro Chiesa, Yuval Ishai, Rafail Ostrovsky, and Omer Paneth. Succinct non-interactive arguments via linear interactive proofs. In Amit Sahai, editor, *TCC 2013*, volume 7785 of *LNCS*, pages 315–333. Springer, Berlin, Heidelberg, March 2013.
- [BCJ⁺24] Foteini Baldimtsi, Konstantinos Kryptos Chalkias, Yan Ji, Jonas Lindstrøm, Deepak Maram, Ben Riva, Arnab Roy, Mahdi Sedaghat, and Joy Wang. zkLogin: Privacy-preserving blockchain authentication with existing credentials. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 3182–3196. ACM Press, October 2024.
- [BCR⁺19] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. Aurora: Transparent succinct arguments for R1CS. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part I*, volume 11476 of *LNCS*, pages 103–128. Springer, Cham, May 2019.
- [BCTV14] Eli Ben-Sasson, Alessandro Chiesa, Eran Tromer, and Madars Virza. Succinct non-interactive zero knowledge for a von neumann architecture. In Kevin Fu and Jaeyeon Jung, editors, *USENIX Security 2014*, pages 781–796. USENIX Association, August 2014.
- [BFS20] Benedikt Bünz, Ben Fisch, and Alan Szepieniec. Transparent SNARKs from DARK compilers. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part I*, volume 12105 of *LNCS*, pages 677–706. Springer, Cham, May 2020.

- [BS23] Ward Beullens and Gregor Seiler. LaBRADOR: Compact proofs for R1CS from module-SIS. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part V*, volume 14085 of *LNCS*, pages 518–548. Springer, Cham, August 2023.
- [CBBZ23] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. HyperPlonk: Plonk with linear-time prover and high-degree custom gates. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part II*, volume 14005 of *LNCS*, pages 499–530. Springer, Cham, April 2023.
- [CGG⁺23] Matteo Campanelli, Nicolas Gailly, Rosario Gennaro, Philipp Jovanovic, Mara Mihali, and Justin Thaler. Testudo: Linear time prover SNARKs with constant size proofs and square root size universal setup. In Abdelrahman Aly and Mehdi Tibouchi, editors, *LATINCRYPT 2023*, volume 14168 of *LNCS*, pages 331–351. Springer, Cham, October 2023.
- [CHM⁺20] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Psi Vesely, and Nicholas P. Ward. Marlin: Preprocessing zkSNARKs with universal and updatable SRS. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part I*, volume 12105 of *LNCS*, pages 738–768. Springer, Cham, May 2020.
- [CMNW24] Valerio Cini, Giulio Malavolta, Ngoc Khanh Nguyen, and Hoeteck Wee. Polynomial commitments from lattices: Post-quantum security, fast verification and transparent setup. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part X*, volume 14929 of *LNCS*, pages 207–242. Springer, Cham, August 2024.
- [DMS24] Michel Dellepère, Pratyush Mishra, and Alireza Shirzad. Garuda and pari: Faster and smaller snarks via equificient polynomial commitments. *IACR Cryptol. ePrint Arch.*, page 1245, 2024.
- [EG25] Liam Eagen and Ariel Gabizon. MERCURY: A multilinear polynomial commitment scheme with constant proof size and no prover FFTs. Cryptology ePrint Archive, Report 2025/385, 2025.
- [FKL18] Georg Fuchsbauer, Eike Kiltz, and Julian Loss. The algebraic group model and its applications. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part II*, volume 10992 of *LNCS*, pages 33–62. Springer, Cham, August 2018.
- [FS87] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *CRYPTO’86*, volume 263 of *LNCS*, pages 186–194. Springer, Berlin, Heidelberg, August 1987.
- [GGPR13] Rosario Gennaro, Craig Gentry, Bryan Parno, and Mariana Raykova. Quadratic span programs and succinct NIZKs without PCPs. In Thomas Johansson and Phong Q. Nguyen, editors, *EUROCRYPT 2013*, volume 7881 of *LNCS*, pages 626–645. Springer, Berlin, Heidelberg, May 2013.
- [GLS⁺23] Alexander Golovnev, Jonathan Lee, Srinath T. V. Setty, Justin Thaler, and Riad S. Wahby. Brakedown: Linear-time and field-agnostic SNARKs for R1CS. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part II*, volume 14082 of *LNCS*, pages 193–226. Springer, Cham, August 2023.

- [GPS25] Chaya Ganesh, Sikhar Patranabis, and Nitin Singh. Samaritan: Linear-time prover SNARK from new multilinear polynomial commitments. In *ASIACRYPT 2025*, volume 16249 of *Lecture Notes in Computer Science*, pages 456–489. Springer, 2025.
- [Gro10] Jens Groth. Short pairing-based non-interactive zero-knowledge arguments. In Masayuki Abe, editor, *ASIACRYPT 2010*, volume 6477 of *LNCS*, pages 321–340. Springer, Berlin, Heidelberg, December 2010.
- [Gro16] Jens Groth. On the size of pairing-based non-interactive arguments. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 305–326. Springer, Berlin, Heidelberg, May 2016.
- [GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive, Report 2019/953, 2019.
- [Hab22] Ulrich Haböck. Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive, Report 2022/1530, 2022.
- [Kil92] Joe Kilian. A note on efficient zero-knowledge proofs and arguments. In *Proceedings of the twenty-fourth annual ACM symposium on Theory of computing*, pages 723–732, 1992.
- [KT24] Tohru Kohrita and Patrick Towa. Zeromorph: Zero-knowledge multilinear-evaluation proofs from homomorphic univariate commitments. *J. Cryptol.*, 37(4):38, 2024.
- [KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In Masayuki Abe, editor, *ASIACRYPT 2010*, volume 6477 of *LNCS*, pages 177–194. Springer, Berlin, Heidelberg, December 2010.
- [Lab17] Protocol Labs. Filecoin: A decentralized storage network. <https://filecoin.io/filecoin.pdf>, 2017.
- [Lee21] Jonathan Lee. Dory: Efficient, transparent arguments for generalised inner products and polynomial commitments. In Kobbi Nissim and Brent Waters, editors, *TCC 2021, Part II*, volume 13043 of *LNCS*, pages 1–34. Springer, Cham, November 2021.
- [LFKN90] Carsten Lund, Lance Fortnow, Howard J. Karloff, and Noam Nisan. Algebraic methods for interactive proof systems. In *31st FOCS*, pages 2–10. IEEE Computer Society Press, October 1990.
- [Lip12] Helger Lipmaa. Progression-free sets and sublinear pairing-based non-interactive zero-knowledge arguments. In Ronald Cramer, editor, *TCC 2012*, volume 7194 of *LNCS*, pages 169–189. Springer, Berlin, Heidelberg, March 2012.
- [Lip13] Helger Lipmaa. Succinct non-interactive zero knowledge arguments from span programs and linear error-correcting codes. In Kazue Sako and Palash Sarkar, editors, *ASIACRYPT 2013, Part I*, volume 8269 of *LNCS*, pages 41–60. Springer, Berlin, Heidelberg, December 2013.

- [LKKO20] Seunghwa Lee, Hankyung Ko, Jihye Kim, and Hyunok Oh. vCNN: Verifiable convolutional neural network. Cryptology ePrint Archive, Report 2020/584, 2020.
- [LMN25] Christophe Levrat, Tanguy Medevielle, and Jade Nardi. A divide-and-conquer sum-check protocol. *IACR Communications in Cryptology*, 2(1), 2025.
- [LPS24] Helger Lipmaa, Roberto Parisella, and Janno Siim. Constant-size zk-SNARKs in ROM from falsifiable assumptions. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part VI*, volume 14656 of *LNCS*, pages 34–64. Springer, Cham, May 2024.
- [LVA⁺25] Hidde Lycklama, Alexander Viand, Nikolay Avramov, Nicolas Küchler, and Anwar Hithnawi. Artemis: Efficient commit-and-prove snarks for zkml, 2025.
- [MBKM19] Mary Maller, Sean Bowe, Markulf Kohlweiss, and Sarah Meiklejohn. Sonic: Zero-knowledge SNARKs from linear-size universal and updatable structured reference strings. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 2111–2128. ACM Press, November 2019.
- [Mic94] Silvio Micali. Cs proofs. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pages 436–453. IEEE, 1994.
- [NMT] Shen Noether, Adam Mackenzie, and Monero Core Team. Ring confidential transactions. <https://lab.getmonero.org/pubs/MRL-0005.pdf>.
- [NS24] Ngoc Khanh Nguyen and Gregor Seiler. Greyhound: Fast polynomial commitments from lattices. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part X*, volume 14929 of *LNCS*, pages 243–275. Springer, Cham, August 2024.
- [PHGR13] Bryan Parno, Jon Howell, Craig Gentry, and Mariana Raykova. Pinocchio: Nearly practical verifiable computation. In *2013 IEEE Symposium on Security and Privacy*, pages 238–252. IEEE Computer Society Press, May 2013.
- [PST13] Charalampos Papamanthou, Elaine Shi, and Roberto Tamassia. Signatures of correct computation. In Amit Sahai, editor, *TCC 2013*, volume 7785 of *LNCS*, pages 222–242. Springer, Berlin, Heidelberg, March 2013.
- [rol21] An incomplete guide to rollups. <https://vitalik.ca/general/2021/01/05/rollup.html>, 2021.
- [SDP22] Nitin Singh, Pankaj Dayama, and Vinayaka Pandit. Zero knowledge proofs towards verifiable decentralized AI pipelines. In Ittay Eyal and Juan A. Garay, editors, *FC 2022*, volume 13411 of *LNCS*, pages 248–275. Springer, Cham, May 2022.
- [Set20] Srinath Setty. Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part III*, volume 12172 of *LNCS*, pages 704–737. Springer, Cham, August 2020.
- [SL20] Srinath Setty and Jonathan Lee. Quarks: Quadruple-efficient transparent zkSNARKs. Cryptology ePrint Archive, Report 2020/1275, 2020.

- [STW24] Srinath T. V. Setty, Justin Thaler, and Riad S. Wahby. Unlocking the lookup singularity with Lasso. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part VI*, volume 14656 of *LNCS*, pages 180–209. Springer, Cham, May 2024.
- [WTs⁺18] Riad S. Wahby, Ioanna Tzialla, abhi shelat, Justin Thaler, and Michael Walfish. Doubly-efficient zkSNARKs without trusted setup. In *2018 IEEE Symposium on Security and Privacy*, pages 926–943. IEEE Computer Society Press, May 2018.
- [XZS22] Tiancheng Xie, Yupeng Zhang, and Dawn Song. Orion: Zero knowledge proof with linear prover time. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part IV*, volume 13510 of *LNCS*, pages 299–328. Springer, Cham, August 2022.
- [XZZ⁺19] Tiancheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, and Dawn Song. Libra: Succinct zero-knowledge proofs with optimal prover computation. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 733–764. Springer, Cham, August 2019.
- [ZCF24] Hadas Zeilberger, Binyi Chen, and Ben Fisch. BaseFold: Efficient field-agnostic polynomial commitment schemes from foldable codes. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part X*, volume 14929 of *LNCS*, pages 138–169. Springer, Cham, August 2024.
- [ZFZS20] Jiaheng Zhang, Zhiyong Fang, Yupeng Zhang, and Dawn Song. Zero knowledge proofs for decision tree predictions and accuracy. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 2039–2053. ACM Press, November 2020.
- [ZSCZ25] Jiaxing Zhao, Srinath T. V. Setty, Weidong Cui, and Greg Zaverucha. Micronova: Folding-based arguments with efficient (on-chain) verification. In Marina Blanton, William Enck, and Cristina Nita-Rotaru, editors, *IEEE Symposium on Security and Privacy, SP 2025, San Francisco, CA, USA, May 12-15, 2025*, pages 1964–1982. IEEE, 2025.
- [ZXZS20] Jiaheng Zhang, Tiancheng Xie, Yupeng Zhang, and Dawn Song. Transparent polynomial delegation and its applications to zero knowledge proof. In *2020 IEEE Symposium on Security and Privacy*, pages 859–876. IEEE Computer Society Press, May 2020.